An analysis of REXX and Python for automation on z/OS.

Vincent Gielen.

Thesis submitted in fulfilment of the requirements for the degree of
Bachelor of applied computer science

Supervisor: Dhr. L. Blondeel
Co-Supervisor: Dhr. B. Tjassens Keiser
Academic year: 2025–2026
First exam period

School of IT and Digital Innovation .

HO
GENT

Preface

The idea for this bachelor's thesis arose from a combination of factors. My interest in automation, and the major focus in the mainframe industry on modernization and the labor shortage, formed the basis of the idea. Additionally, I already had previous experience with Rexx and Python.

First and foremost, I would like to thank Leendert Blondeel. He is the key of the mainframe program at Hogent. He introduced me to the mainframe landscape, and I am very grateful for that.

After I had decided to do a comparison of Rexx and Python, I asked the mainframe community on Discord for advice and ideas. I would like to thank Arthur Coucke. He came up with the idea to use JCL generation in the comparison. After a few emails, Arthur suggested analyzing the entire automation landscape instead of just Rexx and Python. Ultimately, after a conversation with my supervisor, Leendert Blondeel, I decided to keep the comparison solely between Rexx and Python. But instead of executing one large PoC, I wanted to combine several smaller PoCs to conduct a more comprehensive analysis of Rexx and Python. One of those PoCs involves generating JCL.

I would like to thank my co-supervisor, Bobby Tjassens Keiser. While developing my thesis, I could always count on his feedback and expertise. I could rely on Bobby's experience throughout the entire process for feedback on the content as well as for code examples. Through weekly meetings and access to a system to work on, Bobby made this bachelor's thesis possible. I learned a tremendous amount about automation, but also about mainframes in general. This bachelor's thesis would not have been possible without Bobby's help.

Another group of people I certainly must not forget to thank is the mainframe community, both on Discord and LinkedIn. I received a huge amount of input and feedback on every question I asked. The number and quality of responses to my survey also helped me enormously in determining the best requirements and use cases.

I would also like to thank Emma Skovgård for her time and input during the interview, which formed part of the basis for my PoC on RACF.

Finally, I would also like to thank my sister, who greatly helped advance and improve the structure and language of my bachelor's thesis. No artificial intelligence was used to write this bachelor's thesis; instead, I had my sister.

Abstract

This bachelor's thesis concerns automation on the mainframe, more specifically a comparison and analysis of Rexx and Python. Rexx has been present on the mainframe for a long time, whereas Python is a newcomer in comparison. However, due to the skills gap and the development of new technologies outside the mainframe, it is argued that Python could replace Rexx as the most widely used automation language on the mainframe.

The aim of this research is to determine how Rexx and Python compare in terms of automation on z/OS from a technical and practical perspective. To this end, the exact function of automation on the mainframe is investigated, as well as the main reasons for modernizing the mainframe. Subsequently, a comparison of Python and Rexx is made based on predefined use cases and requirements. Ultimately, it is determined whether Python can be a worthy replacement for Rexx.

This research was conducted using a literature study, a requirements analysis, and various proof-of-concepts. For the requirements analysis, several interviews were conducted and a survey was drawn up. In the proof-of-concepts, use cases are executed using criteria determined based on the results of the survey and in consultation with the co-supervisor of this research. The research shows that Rexx and Python each have their own qualities. Python is becoming increasingly popular and is being supported more and more on the mainframe, allowing various Rexx use cases to be adopted. However, Rexx is so intertwined with the tools on the mainframe that it remains indispensable (for the time being). For use cases closely linked to the mainframe system, Rexx is preferred. For use cases requiring communication with the outside world, Python is clearly better. Based on this research, the recommendation for companies is to modernize where possible in order to attract new talent. That talent can subsequently be trained with a mainframe background, including Rexx, and can thus contribute to the maintenance of existing features and the development of new ones on the mainframe. Due to some limitations of the system used and the time available for this research, not all possible use cases or requirements were utilized to compare the languages. With more time and a better system, many more use cases could be executed to conduct a more detailed comparison. Additionally, further research could take a broader view of automation by including other languages and technologies in the comparison.

Contents

List of Figures viii

List of Listings ix

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 Research question . . . 2
- 1.3 Research objective . . . 2
- 1.4 Structure of this bachelor's thesis . . . 3

2 State of affairs 4
- 2.1 What is a mainframe? . . . 4
  - 2.1.1 History . . . 5
  - 2.1.2 Use . . . 5
- 2.2 Automation . . . 5
  - 2.2.1 Automation on mainframe . . . 6
  - 2.2.2 Scripting . . . 6
  - 2.2.3 History of Rexx . . . 7
  - 2.2.4 Other technologies . . . 8
- 2.3 Skills gap . . . 8
  - 2.3.1 Retirement wave . . . 9
  - 2.3.2 Old technology . . . 9
  - 2.3.3 Training programs . . . 9
  - 2.3.4 Organizations/Community . . . 10
  - 2.3.5 Open source . . . 10
- 2.4 Modernization . . . 10
  - 2.4.1 Types of modernization on mainframe . . . 11
  - 2.4.2 Rise of Python . . . 11
- 2.5 Rexx vs Python . . . 12
  - 2.5.1 Advantages of Rexx . . . 12
  - 2.5.2 Disadvantages of Rexx . . . 13
  - 2.5.3 Advantages of Python . . . 13
  - 2.5.4 Disadvantages of Python . . . 14
  - 2.5.5 Overview . . . 14

3 Methodology **15**
3.1 Literature study . . . 15
3.2 Requirements analysis . . . 16
3.2.1 Interviews . . . 16
3.2.2 Survey . . . 16
3.3 Proof-of-Concept . . . 16
3.4 Results . . . 17
3.5 Conclusion . . . 17

4 Requirements analysis **18**
4.1 Interviews . . . 18
4.2 Survey . . . 19
4.3 Selection . . . 20
4.3.1 Use cases . . . 21
4.3.2 Requirements . . . 23

5 Proof-of-Concept **27**
5.1 PoC 1: Using an API . . . 27
5.1.1 Python . . . 27
5.1.2 Rexx . . . 29
5.1.3 Findings . . . 32
5.2 PoC 2: Generating JCL . . . 34
5.2.1 Python . . . 34
5.2.2 Rexx . . . 36
5.2.3 Findings . . . 38
5.3 PoC 3: Using RACF . . . 40
5.3.1 Python . . . 40
5.3.2 Rexx . . . 42
5.3.3 Findings . . . 45

6 Results **47**
6.1 Background . . . 47
6.1.1 Limited comparison . . . 47
6.1.2 System requirements . . . 47
6.1.3 Limitations of use cases . . . 48
6.1.4 Out of scope . . . 48
6.2 PoC Findings . . . 49
6.2.1 Additional findings . . . 49
6.2.2 Requirements . . . 49

7 Conclusion **52**
7.1 Sub-question 1: Why is there a need for the modernization of mainframes? . . . 52

7.2 Sub-question 2: What does automation do on mainframes? 53
7.3 Sub-questions 3 and 4: What are the advantages and disadvantages of Rexx and Python on z/OS? 53
7.4 Sub-question 5: To what extent can Python already replace Rexx? . . . 54
7.5 Conclusion and recommendations 54

8 Interviews **56**
8.1 Interview with Emma Skovgård 56
8.2 Interview with Bobby Tjassens 57

9 Survey **60**
9.1 Rexx vs Python comparative study - success criteria 60
9.1.1 Feel free to share your personal information (name, role, company, ...). This is not mandatory. 60
9.1.2 How long have you been working on Mainframes? 60
9.1.3 What is your experience level with Python (on z/OS)?. 60
9.1.4 What is your experience level with Rexx? 61
9.1.5 When comparing Python and Rexx, what are some requirements that you feel could be used? In other words, what (objective or subjective) criteria can be used to properly compare both? Some examples are (1) dependencies on external libraries or (2) lines of code, ... 61
9.1.6 Feel free to give any sort of comments, feedback, ... that can help me with my thesis. 61

Bibliography **62**

List of Figures

2.1 Table showing the results of the literature when comparing REXX and Python 14

4.1 Image showing the mainframe experience of the people who filled in the survey 20
4.2 Image showing how many people left their name and/or contact details 20

7.1 Table showing the results of the PoC's when comparing REXX and Python 54
7.2 Table showing the recommendation loop for companies 55

List of Listings

5.1 Making contact with a private API on the same mainframe with Python 28
5.2 Making contact with a public API with Python 29
5.3 Making contact with a private API on the same mainframe with Rexx . 31
5.4 Making contact with a public API with Rexx 32
5.5 Python code for using Jinja2 templates to generate JCL 35
5.6 The Jinja2 template used by Python 36
5.7 Rexx code for using ISPF File Tailoring of JCL skeletons 37
5.8 The JCL skeleton used by ISPF File Tailoring with Rexx 38
5.9 The generated JCL file from both the Rexx and Python code 38
5.10 Python code for using RACF 42
5.11 Rexx code for using RACF 44

1

Introduction

Mainframe is one of the oldest technologies from the computer age. Yet it is still used every day by millions of people. The largest companies and financial institutions still rely on that same technology today. Think of a large bank or supermarket chain. They all depend on the stable technology that mainframes offer. Of course, the hardware and software are continuously being improved. For example, AI (Artificial Intelligence) and quantum-safe technology are now already connected to the mainframe. Mainframes are also being modernized by making use of, for instance, APIs (Application Programming Interfaces) and by incorporating UNIX (Uniplexed Information and Computing System) functionality with the help of USS (UNIX System Services). However, the improvement and modernization of mainframe technology does not seem to be reflected in the general perception of mainframes. For years, mainframes have been regarded as an old technology with no future (Sagers et al., 2018). As a result, very few people have specialized in mainframes. Currently, Rexx (Restructured Extended Executor) is the most widely used scripting language on the mainframe (Gilio, 2021). Although Rexx can also be used on platforms other than mainframe, it was written to work perfectly together with mainframes and is therefore also mainly used for that purpose. The largest share of people who can work with Rexx are already employed, and many will soon be retiring. As a result, companies are now facing a major shortage of available workers who can handle that unique technology (Ngo-Ye & Choi, 2018).

1.1. Problem Statement

Although older technologies, such as Rexx, offer stability and are deeply integrated into the mainframe system, they pose a problem for companies due to the shortage of workers. One of the possible solutions to solve the shortage of workers is to offer more mainframe training programs where Rexx is taught, both through

higher education and internally within companies themselves.
Since there are currently very few schools that teach Rexx to their students, companies must, for now, look for another way to attract young talent to mainframes anyway (Blondeel, 2025). A possible other way is to embrace modernization, and make mainframes more accessible and attractive by making use of more modern technologies, such as Python, which offer more flexibility and are better known among new mainframers. Python is a universally known programming and scripting language, even among people with limited IT background. It is therefore the perfect language to reach as many people as possible. On top of that, Python is ideal for modernization, since it opens the door for the mainframe to work together with other systems. According to Gilio (2021), for example, Python is already an important part of modernization on the mainframe today and will also form a large part of future modernization on the mainframe.
However, it is not so straightforward to simply replace scripts written in Rexx, some of which are already 40 years old, with Python, a comparatively much more modern language. Whether Python can simply replace all Rexx code is still unclear, but highly unlikely. It is therefore a difficult decision for companies to choose to simply replace the stability of Rexx with Python, even though that would facilitate the necessary modernization and could attract new workers.

1.2. Research question

This research aims to contribute to a solution for that problem by answering the following research question: How do Rexx and Python compare in terms of automation on z/OS (Zero Downtime Operating System by IBM) from a technical and practical point of view? To answer this question, the following sub-questions must be answered:

- Why is there a need for modernization of mainframes?
- What does automation do on mainframes?
- What are the advantages and disadvantages of Rexx on z/OS?
- What are the advantages and disadvantages of Python on z/OS?
- To what extent can Python already replace Rexx?

1.3. Research objective

The research questions lead to the following research objectives, divided into the problem domain and the solution domain:
Problem domain:

- Determine what the main reasons are for modernization in mainframes.

- Understand what automation does on mainframes and demonstrate how Rexx became the most widely used scripting language in that context.

Solution domain:

- Compare Python and Rexx based on predefined requirements, such as complexity and difficulty of maintenance.
- Determine, through a comparative study and PoCs (Proof of Concept), whether Python can already be a worthy replacement for Rexx, and if so, in which use cases.

The ultimate goal of this bachelor's thesis is to give advice to companies and programmers who are struggling with the choice between Rexx or Python for a given problem. It is impossible to make a comparison for every existing problem. This research attempts to analyze a few common use cases, so that they can provide guidance for working out other use cases.

1.4. Structure of this bachelor's thesis

The rest of this bachelor's thesis is structured as follows:
Chapter 2 describes the current state of affairs, so that the reader gets a good idea of how the problem domain is positioned. In addition, an initial comparison between Rexx and Python is made through a literature study. For this purpose, the advantages and disadvantages of both languages are listed.
Chapter 3 discusses the research methods used in detail. That methodology forms the basis for the practical part of this bachelor thesis, namely the Proof-of-Concept.
Chapter 4 carries out a requirements analysis. This will reveal which success criteria will be used in the Proof-of-Concept in order to make the comparison as accurate as possible. This requirements analysis consists of a literature study, interviews, and a survey.
Chapter 5 presents the most important part of this bachelor thesis, namely the Proof-of-Concept. In the PoC, Rexx and Python will be compared using several commonly used use cases for systems programmers. The use cases and success criteria are determined with the help of findings from the requirements analysis.
Chapter 6 summarizes the results and findings from the PoCs. It also reflects on the scope of the research and the significance of the results within it.
Finally, Chapter 7 presents the conclusion and formulates an answer to the research questions. In addition, suggestions are given for future research within this domain.

2

State of affairs

This chapter attempts to sketch a picture of the current state of the mainframe industry with regard to automation and modernization. It also discusses an already existing comparison between Rexx and Python. Following is a brief explanation of the history and use of mainframes. After that, a deeper look is taken at automation and scripting, and which technologies can be used for that on mainframe. Next, the issue of the skills gap in the mainframe sector is discussed, and how modernization can be a possible solution to this. Finally, a summary is given, based on existing literature, of the advantages and disadvantages of Rexx and Python that have already been demonstrated.

2.1. What is a mainframe?

When this bachelor's thesis refers to mainframe, it means the IBM Z Mainframe. Although mainframes are usually associated with IBM (International Business Machines), it is important to clarify that they are not the only company that offers mainframes. However, this bachelor's thesis is limited solely to the IBM Z Mainframe. That does not mean that the conclusions from the bachelor's thesis cannot also apply to other mainframes. But these other companies have not been taken into account when writing this bachelor's thesis.
In an article by IBM, Susnjara and Smalley (2024) describe mainframes as follows:
'At their core, mainframes are high-performance computers with large amounts of memory and data processors that process billions of simple calculations and transactions in real-time. A mainframe computer is critical to commercial databases, transaction servers and applications that require high resiliency, security and agility.'

2.1.1. History

The history of mainframes goes back a very long way. Over the years, many changes have been made to the architecture of mainframes. One example of this is the ability to virtualize. This ensures that hardware can be used more efficiently. With every new mainframe generation, numerous new improvements and new features are released (Jacobi & Webb, 2020). In this way, IBM ensures that their machines can meet the continuously evolving demands of their customers. Things such as fraud detection with the help of artificial intelligence or quantum-safe technology, where data is protected as a precautionary measure against the rise of quantum computers, have been available since the IBM z16 (the name of the mainframe machine) (Ngo-Ye & Choi, 2018; Stine, 2022).
In addition to the evolution of the hardware and the development of new features, there is also an evolution in how mainframers deal with the technology, and with new technologies outside of the mainframe. For example, the OMP (Open Mainframe Project) conducted a comparative study of three mainframe-oriented programming languages against three more modern programming languages (Open Mainframe Project Modernization Working Group, 2024). This showed that some modern languages offer the ability to replace older languages. There are also various mainframers who promote the use of more modern technologies and point out the advantages this can bring (Gilio, 2021). This evolution of the mainframe toward the use of modern programming languages and technologies, also outside of the mainframe, is called the modernization wave. This is discussed further later on.

2.1.2. Use

Mainframes are mainly used by large companies, such as banks, airlines, and wholesalers. These companies process enormous amounts of data, and must correctly process a huge number of transactions every day. An estimated total of 30 billion transactions are carried out daily on mainframes, spread across the thousands of companies that use mainframes (Ngo-Ye & Choi, 2018). The reason mainframes are used so extensively by these large companies is that mainframes, due to their unique architecture and construction, keep running almost continuously. Both during the busiest Christmas periods and at three o'clock in the morning, people need to be able to carry out payments.

2.2. Automation

Automation (research) can be described as follows:
"Automation research emphasizes efficiency, productivity, quality, and reliability, focusing on systems that operate autonomously, often in structured environments over extended periods, and on the explicit structuring of such environments." (Gold-

berg, 2012, p. 1)
Automation is not only used in IT, but it is very present there. This is because, with the help of automation, manual tasks can be carried out by a program, which frees up more time for other matters. Automation has certainly become important with the rise of virtualization. We speak of virtualization when a server (typically an x86 server) is split into several smaller servers, in order to make optimal use of the available resources. To set up each new smaller server with its own system, the same commands would need to be executed for each server every time. To avoid this, that process can be automated. To ensure that this happens in a high-quality and reliable manner, one can create scripts that use logic to handle various scenarios that may occur. This is just a simple example, but it gives an idea of the possibilities of automation. (Interview with Bobby, 8.2)

2.2.1. Automation on mainframe

Since mainframes are a few years older than typical consumer-oriented computers, it is not illogical that automation on the mainframe also had a certain head start. When talking about automation on the mainframe, one thinks first and foremost of so-called "batch jobs." Powell and Smalley (2024) describe batch jobs as 'any regularly occurring automated process that groups similar tasks and performs them automatically without needing to be prompted by human interaction'. Powell explains that there are major advantages to using batch jobs. One of those advantages is reducing errors made by users, because interaction with users is reduced to a minimum. Another advantage of batch jobs is that they can be run at a time ideal for the company. Such batch jobs are usually run at the beginning or end of the working day, depending on their function. Batch jobs are also often run at night, when the system otherwise expects minimal user input. Think of a bank that is continuously busy with all kinds of customer-oriented matters throughout the day. That bank can then run batch jobs at night, when most people are not making payments, without customers noticing any delay in the system due to the high workload that batch jobs require. To schedule such batch jobs, also called scheduling, JCL (Job Control Language) is widely used on the mainframe (Interview with Bobby, 8.2). JCL will be discussed further later on in one of the use cases of this paper.
In addition to batch jobs and JCL, scripts are also written on the mainframe to automate routine tasks. Examples of this include creating users, assigning permissions, handling log messages, and more. Writing these scripts is largely done via JCL and REXX, depending on the desired function (Interview with Bobby, 8.2).

2.2.2. Scripting

JCL, Rexx, and Python are all scripting languages. Scripting is a particular form of programming. A script functions mainly as a kind of glue between other pro-

grams rather than adding much functionality itself. When writing a script, one assumes that programs with certain functionalities already exist, which one wants to connect to each other. For example, Rexx can call a DB2 (database for IBM Z Mainframes) to filter certain data from a table. That data is then placed into a new dataset, to be later emailed to various people. All these things, such as the database, the creation of a new dataset, and the mail function, are functionalities that exist outside of Rexx. Rexx is the glue that allows those functionalities to work together. Ousterhout (1998) explains that scripting makes use of a different way of writing code, without types, which makes it simpler to connect different components to each other, with fewer restrictions. He predicted that the popularity of scripting would only increase in the future. This was because enormous progress had already been made in terms of languages. In addition, the improved speed of computers would ensure that the slower scripting languages would get more opportunities, because the difference in speed compared to other more efficient system languages, such as assembler, would have less impact on the overall speed of a program. Whether these are indeed the two reasons for the growth of scripting lies outside the scope of this research. But that scripting and automation have grown is certain. As a final argument, Ousterhout (1998) states that the learning curve of a scripting language is fairly small due to the often fairly simple syntax. This ensures that a larger community emerges, which makes the scripting languages even better and more popular. That is a phenomenon that has occurred with Python, a modern scripting language, which will be discussed further later in this chapter (Lindstrom, 2005).

2.2.3. History of Rexx

Rexx started as a hobby project of Mike Cowlishaw in 1979. Rexx was originally created to replace the even older EXEC and its successor EXEC2. Those languages were among the first automation languages. EXEC2 was used to bundle various commands into a script, so that the programmer did not have to enter these commands manually. Initially, these scripts contained a small amount of logic. Logic in a script ensures, for example, that certain commands are or are not executed, depending on whether earlier commands were executed successfully or not. That logic became larger and more complex over the years. This is where the limits of EXEC2 became apparent. The use of the language became too large for what it was made for (M. Cowlishaw, 1994). Part of that limit was also the readability of the code. Because little logic was initially expected in the code, certain syntax choices were made that, although perfectly readable by the computer, were not so easily readable by programmers. The intention of Rexx was to create a new scripting language that was both powerful and user-friendly, so that both of the above limitations would be resolved. Some examples of how the language was made more user-friendly are, for instance, the relatively small instruction set, the

case-insensitivity, and the absence of special characters. Those small tweaks ensure that the language is easy to learn, that programmers do not constantly have to worry about syntax, and that it is much easier to understand and maintain someone else's code (M. F. Cowlishaw, 1984). Rexx quickly became popular, and thanks to feedback the language could be quickly adapted to the needs of users. That was, after all, Cowlishaw's goal: to create a scripting language that was, first and foremost, user-friendly. Since 1982, Rexx has continued to evolve. It is available on almost all operating systems and a Rexx symposium is still organized every year. The language is still widely used, mainly on mainframes. Although studies such as that of Winkler and Flatscher (2023) suggest that Rexx is indeed user-friendly and easier to learn, other studies such as Open Mainframe Project Modernization Working Group (2024) point to the fact that there is little community support for Rexx. That, combined with continuously evolving technologies both within and outside the mainframe, is causing Rexx to decline in popularity and number of users.

2.2.4. Other technologies

Thota (2020) describes in her paper the rise of the cloud, and its growing popularity. She argues that it is therefore necessary for companies to also adapt their strategy regarding the automation of processes to the changing demands of the infrastructure. She suggests the use of, among others, Terraform, Ansible, and Kubernetes in the cloud. Although the cloud and mainframe are two different systems, the suggested technologies could also be considered for taking on certain automation tasks from the mainframe. However, discussing those technologies would require an entirely new research project. Therefore, they are not discussed further in this research, but it is recommended in the conclusion that further research also address these new technologies.

2.3. Skills gap

One of the biggest challenges that organizations with mainframes face is finding suitable personnel who can/want to work with mainframes. There is a gap between the number of people specializing in mainframes and the number of experts needed to maintain all the mainframes. This is often called the mainframe skills gap and has existed in the mainframe world for years. In 2005, Barnett (2005) spoke about the shortage of mainframe skills, and why this could be a potential concern for companies. There are several other studies that have reached the same conclusion over the years (Murphy et al., 2010; Ngo-Ye & Choi, 2018; Vanaparthi, 2025). Everyone seems to agree that there is a shortage of mainframers. In the following subsections, the causes of this shortage are addressed, followed by a discussion of initiatives to reduce the skills gap.

2.3.1. Retirement wave

A first reason why there is a shortage of mainframers is that a generation of mainframers, the baby boomers, are retiring (Phillips et al., 2013). This generation has gained years of experience and expertise, but there is a lack of a next generation to take over that expertise. This carries the enormous risk that important information and skills will be lost (Sagers et al., 2018).

2.3.2. Old technology

Another possible reason for the shortage of mainframers is that the mainframe has the reputation of being an old technology. The disappearance of the mainframe has been announced several times already. A well-known prediction was that of Stewart Alsop, when in 1991 he claimed that the last mainframe would be shut down five years later. That prediction, of course, did not come true, and many similar predictions in the years that followed did not either. Many companies that use mainframes also do not seem to be planning to get rid of their mainframes any time soon (Barnett, 2005). Nevertheless, the mainframe has never lost its reputation as an old or outdated technology, which can influence the choices of IT professionals to specialize in other areas of IT besides mainframe.

2.3.3. Training programs

On top of the fact that a large portion of the people working with mainframes will soon be retiring, and that the mainframe has an unfortunate reputation, universities also do not sufficiently respond to the growing demand for mainframers (Sagers et al., 2018). There are a few universities that do offer a mainframe curriculum, such as Northern Illinois University and Hogent (Blondeel, 2025; University, 2026). However, those universities are in the minority (Sochova & Kunce, 2012). Hogent is even the only university in Europe that offers a mainframe course (Damme, 2025).

There are various initiatives that try to compensate for the limited availability of training programs by offering courses, certificates, and so on. In this way, an attempt is made to make the big step toward a mainframe career easier. IBM itself offers various courses, and a whole range of badges and certificates for people who take certain courses. In this way, they ensure that people become, and remain, motivated to take mainframe courses. IBM has created a course specifically for beginners, called Z Xplore (IBM, 2026). In addition, there are other companies that also contribute to mainframe courses, and even build a business model around it. A good example of this is Interskill. Interskill is a company that specializes in offering mainframe training, and their courses are offered to many juniors. They provide manageable training programs to quickly and efficiently learn a lot about mainframes. Interskill is responsible for a large portion of the IBM badges that are earned, about 80% in 2024 (Learning, 2024).

2.3.4. Organizations/Community

In addition to official training programs and courses from commercial companies such as IBM and Interskill, there are also other organizations that try to pass on mainframe knowledge to a new generation. Organizations such as GSUK, SHARE, or Mighty Mainframe organize annual events where hundreds of experts from all over the world gather at the same location. They come to speak about their expertise, new findings, or technologies, and give the younger generation the opportunity to ask their questions (M. O. E. Project, 2025).
Thanks to these events, and because the mainframe world is fairly small, many people know each other, and a mainframe community has developed over the years. This community has grown significantly since the emergence of a Discord server, System Z Enthusiasts, where everyone with an interest in mainframes is welcome. This makes contact between experienced mainframers and inexperienced beginners easier, and allows mainframe knowledge to be shared smoothly among people all over the world (Perva, 2024).

2.3.5. Open source

Finally, there are also various initiatives, from both large companies and individuals, to contribute to open source projects for mainframe. Open source means that the code for certain applications is freely available for others to use, to modify, to add things to, and so on. Open source means much more than that, but for the scope of this research it is sufficient to know that contributing to open source code ensures that a larger group of people has (free) access to certain code, and that people from different companies and organizations can collaborate to improve mainframe environments. (Perens et al., 1999)
Some examples of open source projects are SEAR, Zowe, and many more projects that can found on the OMP website [1].
Despite the many initiatives, currently it does not seem that all these initiatives will bring enough change to replace the people who are retiring with new mainframe experts, in time.

2.4. Modernization

In the previous chapter, the skills gap was discussed, along with the initiatives being taken by various institutions to reduce that gap. In this chapter, another initiative is discussed, namely the modernization of the mainframe. Modernization may perhaps be a better solution to several problems at once. It would make it easier (1) to respond to technological progress being made outside of the mainframe platform and (2) to make the leap to a mainframe career smaller for those who have experience in other branches of IT. In addition, there are many other advantages, such

[1] https://openmainframeproject.org/

as cost savings and cloud integration. The latter are mentioned in this chapter, but not discussed in detail.

2.4.1. Types of modernization on mainframe

With the rise of new technologies outside of the mainframe, and the shortage of experienced mainframers, some companies feel compelled to modernize their old mainframe systems (Barnett, 2005). The costs associated with mainframes are also a motivation for companies to look at other options (Vanaparthi, 2025). What exactly this modernization entails can depend on various factors, and is unique to each company. Barnett (2005) describes four categories of mainframe modernization:

- Access/extension: a new interface that can communicate with the mainframe.
- Integration: a more in-depth way of allowing communication between the mainframe and other platforms.
- Re-engineering: moving parts of a system/code from the mainframe to another platform.
- Platform migration: completely moving a system from the mainframe platform to another platform.

In some cases, modernization can thus mean that entire systems are removed from the mainframe and placed on another platform, usually the cloud (Zlatanov, 2016). Although this is generally viewed critically, some successful migrations have already taken place (Khadka et al., 2015; Spain, 2025). Cloud companies can therefore benefit enormously from the modernization of mainframes, and even IBM is responding to this with its own zCloud (Ngo-Ye & Choi, 2018). Zlatanov (2016) argues that the use of the cloud is a logical evolution, because the focus of companies has shifted over the years from performance to convenience. Khadka et al. (2015) and Vanaparthi (2025) show that the benefits go further than just convenience. Cost savings, user-friendliness, and maintainability are other reasons why companies migrate from the mainframe to cloud.

2.4.2. Rise of Python

Although the arguments for a (partial) migration from the mainframe to the cloud are strong, there are also plenty of counterarguments, such as complexity and project costs (Ngo-Ye & Choi, 2018). Mainframes also have a high degree of customization, whereas the cloud is largely the same for everyone, which also brings risks (Zlatanov, 2016). Therefore, there are also other modernization strategies where a migration to another platform is not strictly necessary. The use of new technology from outside of the mainframe was discussed earlier. The use of these technologies is also possible from the mainframe itself, with the help of tools

developed by various companies and organizations. The OMP organization was mentioned earlier as one of the organizations that reduces the knowledge gap for newcomers to the mainframe. Through the development of tools such as Zowe - which makes interaction with the mainframe easier - OMP plays a central role in the process of making mainframes more user-friendly for newcomers (Docs, 2026; O. M. Project, 2025). In addition, IBM itself also develops tools, such as ZOAU (IBM Z Open Automation Utilities), which make it possible to use modern technologies on the mainframe. ZOAU works very well together with Zowe and makes it possible to use, among other things, Python from the mainframe (Singh, 2025). Since its introduction on the mainframe, Python has often been cited as an example of mainframe modernization. The possibilities of Python are enormous. This is also evident from the interview with Emma (8.1), in which she describes how she uses Python to automate parts of her job and does all new functionality with Python. Emma is also one of the developers of the SEAR (Security API for RACF) project[2], which contributes to the modernization of RACF (Resource Access Control Facility), IBM's security software product for mainframes, by making it available for more modern languages than Rexx, including Python. It thus appears that Python may possibly take over many functions from Rexx. In the following chapter, a deeper look is taken at the precise advantages and disadvantages that both languages have.

2.5. Rexx vs Python

This chapter presents a comparison of Rexx and Python, each with advantages and disadvantages. This forms the basis for the further comparisons of both languages in the proof-of-concept. Two studies have produced conflicting results regarding the performance of the languages. Since these claims cannot be tested in this research, that part of the comparison will not be discussed further (Prechelt, 2000; Sadavarte & Prabhu, 2022).

2.5.1. Advantages of Rexx

As mentioned in the chapter on the history of Rexx, Rexx was written in 1979 by Mike Cowlishaw with the main goal of being user-friendly. This user-friendliness is a major asset of Rexx, because it ensures that people can easily understand and maintain each other's code (Fosdick, 2005). In addition, the Rexx language is also fairly small in terms of syntax, which makes it very easy to learn and to work with almost immediately (Winkler & Flatscher, 2023). Winkler also explains that very few syntactical errors can occur in the code thanks to the case-insensitivity - Rexx can be written in both upper and lower case - and dynamic data types of Rexx. That makes Rexx a very pleasant language to learn. Specifically for mainframe, Rexx is

[2] https://github.com/Mainframe-Renewal-Project/sear/tree/main

one of the best and most widely used scripting languages. Partly because Rexx was specifically made to work on mainframes, mainframes come with support for Rexx by default. Rexx therefore has access to, and can easily work together with, a great many aspects of the mainframe. Rexx is also available on other platforms besides the mainframe, but that is not relevant to this research.

2.5.2. Disadvantages of Rexx

One of the advantages of Rexx is also a first disadvantage, namely that the language has a small syntax. That makes it easier to learn, but it also means that the functionalities of Rexx are limited, especially compared to Python. Rexx does not have extensive public resources that give the language more functionalities (Sadavarte & Prabhu, 2022). The positive robustness of Rexx is thus also one of its biggest disadvantages. The second disadvantage of Rexx is the lack of an extensive community. This means there are few people who can work fluently with Rexx. This is not due to the language being too complicated to learn, but due to the fact that other languages, such as Python, are more popular among the newer generation of IT professionals. This is also shown by the interview with Emma (8.1). This may be due to the fact that there are far more resources available on the internet for learning Python than Rexx (Open Mainframe Project Modernization Working Group, 2024). The last disadvantage of Rexx brings us back to the modernization of the mainframe. Although possible, it is challenging to use Rexx for a lot of modern technology, such as the use of APIs or working together with modern cloud-based environments and principles (Open Mainframe Project Modernization Working Group, 2024). The three mentioned disadvantages reinforce each other and make it a challenge to use only Rexx while keeping up with the continuously evolving technology landscape.

2.5.3. Advantages of Python

Many of the previously mentioned disadvantages of Rexx are advantages of Python. Python is enormously popular, so there is no question of a shortage of workers (Sanglier et al., 2022). Thanks to its popularity, Python has an abundance of external libraries that, if Python's core functionalities are not enough, can often provide a solution to a problem. There is such an abundance that it is recommended to first search for an existing Python library before starting to write one's own code to solve a problem, in order to avoid reinventing the wheel (Lindstrom, 2005). There is also an abundance of resources for learning Python. According to Emma (8.1), Python is often one of the only languages that is also taught to non-IT people, which shows that no extensive background knowledge is needed to master the language. Finally, Python can work together with all kinds of modern technology without any problems, and it opens the door to quickly responding to changes in the future (Gilio, 2021).

2.5.4. Disadvantages of Python

The popularity of Python may explain why few articles can be found that discuss the disadvantages of Python. Most articles seem to suggest that Python is slower than other languages that are closer to the operating system, such as Rexx. However, as mentioned earlier, this cannot be verified in this research. The interview with Bobby (8.2) and a blog by Redwerk (2024) indicate that a major disadvantage of Python is that problems can arise due to dependencies. Because of the many libraries and packages that make Python so good, there can sometimes be overlap within a company. If one team works with one version of a package and passes code on to another team that is working with a different version, conflicts can arise that are annoying to resolve. In addition, external packages can also pose security problems, since the underlying code is written by someone else. Emma (8.1) explains that this can be avoided through sufficient knowledge on the part of programmers, and by focusing on the core functionalities of Python, which by themselves already far exceed the functionalities of Rexx. The interview with Bobby and Emma (8.1,8.2) also showed that Python does not have the same access to the mainframe as Rexx. Via ZOAU, Python can communicate reasonably easily with the mainframe, but that still does not come close to the level of integration that Rexx has with mainframe functionalities.

2.5.5. Overview

Figure 2.1 gives an overview of a number of elements on which Rexx and Python do or do not score well. The checkmarks only indicate that there is a clear winner, not that the loser is entirely lacking the mentioned functionality. This table serves only as a handy overview of the advantages and disadvantages mentioned in this section.

Category	REXX	Python
Integration with tools on the mainframe	✓	×
Integration with tools outside the mainframe	×	✓
User-friendliness	✓	✓
Standard functionalities	×	✓
People who know the language	×	✓
Dependencies	✓	×

Figure 2.1: The results of the literature when comparing REXX and Python.

3

Methodology

To answer the research question, five sub-questions were answered: (a) Why is there a need for modernization of mainframes?; (b) What does automation do on mainframes?; (c) What are the advantages and disadvantages of Rexx on z/OS?; (d) What are the advantages and disadvantages of Python on z/OS?; (e) To what extent can Python already replace Rexx? The five sub-questions were approached through two research techniques: a literature study and a comparative study. The research techniques consist of various steps that were divided into three phases. The first phase was the literature study, which provided an answer to the first two sub-questions, and laid the foundation for sub-questions 3 and 4. The second phase was a requirements analysis. This helped with (1) identifying commonly used functionalities and use cases with which Rexx and Python can be compared; (2) criteria that both languages must meet. Finally, in the third phase, the PoCs for each use case followed, which together provided more insight into the advantages and disadvantages of both languages (sub-questions 3 and 4), and which provided an answer to the question of whether Python can already replace Rexx (sub-question 5).

3.1. Literature study

In the literature study, an attempt was made to give an overview of the current situation in the field of mainframe. To do this, the past was first examined, and how Rexx became such a major scripting language on mainframe. After that, the rise of Python and its popularity was examined. Finally, an analysis was made of existing research comparing the two languages. This comparison, which showed that both languages have their advantages and disadvantages, formed the basis for the further research in this bachelor's thesis. In the literature study, the first two sub-questions (Why is there a need for modernization on mainframes? & What

does automation do on mainframes?) were fully answered. In addition, the third and fourth sub-questions (What are the advantages and disadvantages of Rexx on z/OS? & What are the advantages and disadvantages of Python on z/OS?) were also partially answered already. However, that latter comparison in the literature was mainly used as support in determining the requirements needed for a successful proof-of-concept. The results of this phase can be found in Chapter 2.

3.2. Requirements analysis

A requirements analysis was carried out to determine which commonly used functions and use cases were used to make the comparison between Rexx and Python. In addition, in this phase the success criteria against which both languages were tested were determined, in order to reach a conclusion about the best option for each use case. To determine the best possible use cases and success criteria, several interviews were conducted with experienced systems programmers, and a questionnaire was drawn up to obtain broad input from people in the field. Afterward, with the help of the MoSCoW method, a clear picture was formed of the requirements that both languages must meet in order to succeed in a use case. In this way, the preference for one of the two languages in that specific use case was determined.

3.2.1. Interviews

The interviews were conducted with experienced systems programmers. The information from the interviews was used to get a good picture of how mainframers experience modernization on the mainframe, as well as to obtain input on the use cases and requirements that were chosen for the PoC.
The transcribed interviews are included in Appendix 8.

3.2.2. Survey

The survey was drawn up with the goal of obtaining input from as many systems programmers as possible, with a low barrier to entry. The success criteria in particular were surveyed, since there can be different opinions about these and this research aims to carry out an objective analysis. The survey was shared in the Discord community of 'System Z Enthusiasts' and on LinkedIn. The survey can be found in Appendix 9.

3.3. Proof-of-Concept

In the Proof-of-Concept, the comparative study was carried out. Several smaller PoCs were drawn up based on the previously determined use cases and requirements. Various use cases were worked out, in order to get a broad picture of the differences between the two languages, each with their strengths and weaknesses.

For each PoC, an argument was made as to why this is an important use case, and what the success criteria are that a solution must meet. In addition, for each PoC the code used was shown and explained. And the findings were also explained on the basis of the predetermined success criteria.

3.4. Results

In the results, reflection was given on the scope of the research and some limitations that may have influenced the results of this research. In addition, the findings from the various PoCs were combined and analyzed, and reflection was given on which use cases could be carried out in future research.

3.5. Conclusion

In the conclusion, the research questions were revisited based on the findings and results of the research. A reflection was also added on the results with a view to the broader mainframe ecosystem and its future. Based on this, advice was formulated for companies regarding their approach to automation.

4

Requirements analysis

In the requirements analysis, two questions are answered. First, it is determined which use cases or functions will be carried out in the PoCs to compare Python and Rexx with each other. Second, it is determined which success criteria will be used to compare the two languages in the execution of a use case.
Various methods are applied to obtain these requirements. Two interviews are conducted. In addition, a survey is drawn up, in order to obtain input from the mainframe community. The input from the co-supervisor is also used to make the final decision about which requirements will ultimately be used.

4.1. Interviews

For this research, two interviews were conducted. In addition, individual conversations with various mainframers provided a lot of input and added value for this research.
One of the interviews is with Emma Skovgård. That interview can be found in 8.1. Emma works as an 'Information Security Professional' and is known as a big Python enthusiast for the z/OS platform. She is also very active in the mainframe community, both on LinkedIn and in the Discord community. A large portion of the people currently working with mainframes have a preference for the tools that have been the standard for years. This is why this interview provided great added value. It made it possible to view the current state of affairs and the future of the mainframe through the lens of someone who will probably still be working with it for quite a while. This interview made a major contribution to the chapter on the state of affairs and the initial comparison between Rexx and Python, and also provided input for the use cases and requirements. Emma also filled in the survey, so her input is reflected there as well.
In addition, several sessions were organized with the co-supervisor of this bachelor

thesis, Bobby Tjassens. Bobby is an experienced system programmer and is also very active in the community. One of those sessions is considered an interview because it mainly discussed information and knowledge about the history of the mainframe and automation. This interview, in 8.2, therefore mainly contributes to the chapter on the state of affairs. The other sessions provided indispensable input in determining the use cases and requirements, and in carrying out the PoCs.
Both Emma and Bobby are (co-)developers of the previously mentioned open source project SEAR. SEAR is discussed further later in this research, in one of the use cases.

4.2. Survey

The purpose of the survey was to obtain as much input as possible from the mainframe community. This is a good source because the results of this research are also intended for that target group. In addition, the community is so small and helpful that this seemed like a good way to get discussions going and thereby obtain even more input. At the time of writing and processing the results, the survey had been filled in 22 times. In addition, there was also a lot of response to the LinkedIn post promoting the survey. All those responses together ensured that more than enough requirements and use cases were collected to choose from for this research. The survey is included as an appendix in 9.
Before the results of the survey can be analyzed, it is essential to look at the demographic characteristics of the respondents. If those are too one-sided, for example only students or people with a strong language preference or anonymous participants, this would limit the value of the results. In figure 4.1, the demographic characteristics are depicted. From these figures it can be deduced that the general level of experience with the mainframe is fairly high. The same applies to experience with Rexx. These two can, of course, be linked to each other. The experience with Python is considerably less. A possible reason for this could be that a large portion of the respondents are experienced mainframers who do not yet have much experience with newer languages such as Python. Another possible reason could be that Rexx is a language that virtually every mainframer must know, whereas Python is (so far) an optional language for working with mainframe. These are merely speculations, so no further conclusions are drawn from these figures regarding this. It does clearly show that the experience level is high enough, especially in terms of mainframe and Rexx, to consider the given requirements credible. The last figure (4.2) shows how many respondents left their name and/or contact details. Although this does not guarantee the value and credibility of their answers, it can provide an indication. If someone leaves their name, it can be verified whether the answers come from a real person, and their background can be examined to see whether it matches the given answers. Although it is still possible that someone provides a false name, this research assumes - given how small and helpful the mainframe

community is - that almost all answers come from real people with good intentions. In the worst case, one can fall back on the experience of the co-supervisor of this research to verify the credibility and feasibility of certain use cases and requirements.

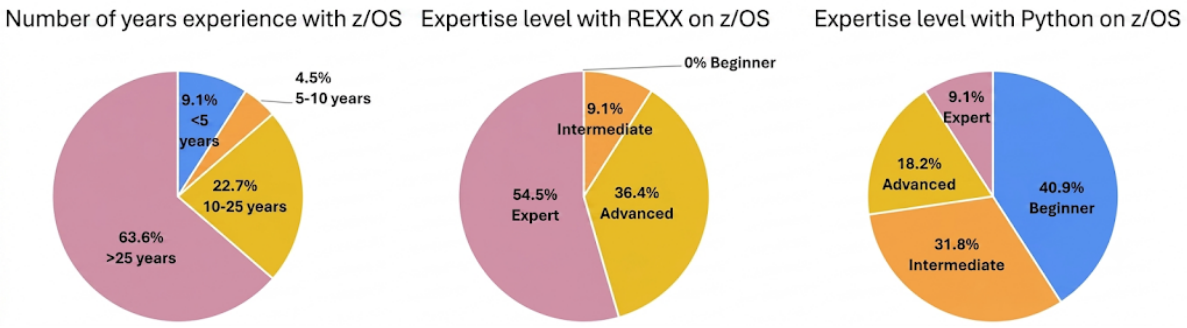

Figure 4.1: The experience level with mainframes, Rexx, and Python of the survey respondents.

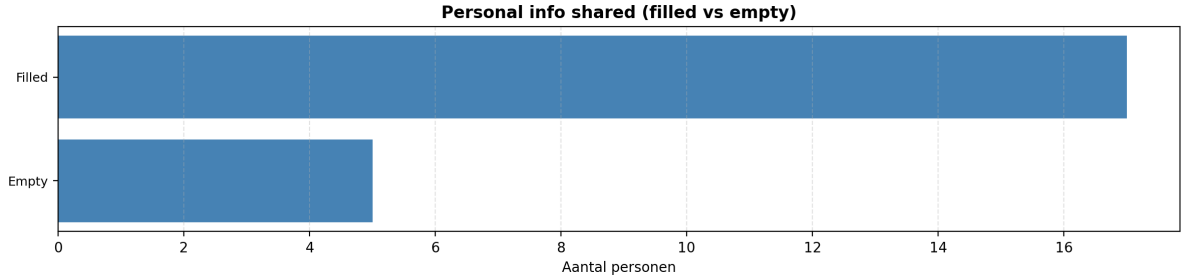

Figure 4.2: The number of people who left their details on the survey.

4.3. Selection

As mentioned, quite a lot of use cases and success criteria were collected from the survey and interviews. From this collection, a selection was made. That selection was made based on various criteria. For both the use cases and success criteria, it must be possible to carry them out without access to a production system. This means that anything related to CPU usage, speed, or general performance is not possible. In addition, some subjective success criteria, such as readability and learnability, were not included in the final selection. This is partly because the literature study showed that both Rexx and Python score fairly well on these aspects, but also because such subjective criteria are difficult to test. In practice, it is probably the case that a combination of objective and subjective criteria will influence the choice for one language over the other. Finally, a few use cases that fall outside the scope of the current comparison were also left out. In the following subsections, an overview is given with explanations of the use cases and requirements.

4.3.1. Use cases

Below is an overview of the various use cases that are carried out in this research, as well as some use cases that were not carried out. In each case, it is explained why a use case was (not) selected. The selected use cases were chosen based on the survey results, and in consultation with the co-supervisor of this research. The use cases must meet various criteria, so that they are executable on a non-production system, and so that they are not so complex that a student cannot carry them out and analyze them.

Selected

The use cases that will be carried out as PoCs are the following:

- **Using an API:** An API (Application Programming Interface) is often used as the connecting link that enables different systems to communicate with each other. Such communication takes place using REST (Representational State Transfer), which makes use of HTTP (Hypertext Transfer Protocol) and methods such as GET, PUT, POST, and DELETE. Those methods form the basis of the communication between two systems and help determine which data is sent from one side to the other. That data is usually sent in the JSON (JavaScript Object Notation) format. That is a format that can be read by both machines and humans. In this way, a system can send a standardized method with data to another server, which in turn sends back the necessary response. APIs are essential to the functioning of the internet and for contacting systems outside one's own system. In the specific case of the mainframe as well, APIs are being used more and more. Although the mainframe can run on its own, nowadays it needs to be able to form part of a broader ecosystem. The mainframe must be able to communicate with the outside world for all kinds of crucial tools that exist outside the mainframe. It is therefore essential to be able to reach an API efficiently from the mainframe. For Python this is an easy task, since there is the widely used requests package, which can send standardized requests via HTTP. For Rexx it is less straightforward. For Rexx there is the z/OS Client WET (Web Enablement Toolkit) which can also make such requests. In a simpler way, a cURL (Client URL) can also be manually constructed in Rexx through string manipulation, which can be used to send the API requests. Depending on the complexity of the use case, one may be preferred over the other. This is further explained in the PoC.
- **Generating JCL via skeletons:** JCL (Job Control Language) is the language used on a mainframe to define and execute batch jobs. A JCL job describes which programs need to be executed, which datasets are used in the process, and how the system should handle those datasets. JCL is therefore essential to the functioning of the mainframe, but has the disadvantage that it can be very repetitive. Many jobs have a similar structure, where only a number of

parameters, such as dataset names or attributes, differ. For system programmers or DB2 administrators, it can save an enormous amount of time and effort if that process is automated. JCL skeletons are often used for this purpose. A skeleton is a JCL template in which variables are placed at the points where the values may differ. Those variables are filled in with the correct values at the time of execution, so that correct and consistent JCL is generated each time without the user having to manually write the full JCL. This can be done via ISPF (Interactive System Productivity Facility) File Tailoring, a mechanism that processes skeletons and substitutes the variables. This ISPF File Tailoring can in turn be invoked via Rexx. A similar result can be achieved with Python and the Jinja2 package, where Jinja2 takes on the same role of template processing.

- **Using RACF:** RACF (Resource Access Control Facility) is the security system used on a mainframe to manage access to resources. RACF determines who has access to which datasets, programs, and system functions, and keeps track of when and how that access takes place. The system works on the basis of user profiles, groups, and resource profiles, whereby for each resource it is explicitly defined which users or groups have access to it. RACF is therefore essential to the security of the mainframe, but managing it can be very extensive. Large organizations can have thousands of users and resources, where manually managing all those profiles is time-consuming and error-prone. For systems administrators, it can therefore save an enormous amount of time and effort if that management is automated. This can be done via TSO (Time Sharing Option) commands such as `ADDUSER`, `ALTUSER`, and `LISTUSER`, which can be invoked directly from REXX. Those commands allow user profiles to be created, modified, and queried. A similar result can be achieved with Python via the SEAR API, where SEAR takes on the same management tasks but through a modern REST interface.

Not selected

These are some of the most frequently proposed use cases that were not chosen for this research. For each of the use cases, it is briefly explained what they involve and why they were not included in this research.

- **Starting up an LPAR (Logical Partition):** This use case would have been a perfect example of how Rexx has the upper hand over Python when it comes to matters deeply integrated into the system. When starting up an LPAR, among other things, Rexx scripts are fired off. The idea of this use case would be to replace those Rexx scripts with Python. However, with the limited access to the system, it is impossible to carry out this use case.
- **Translating PARSE & ADDRESS in Rexx to Python:** Various commands in Rexx are enormously powerful, and work well together with the mainframe

system. It is therefore important to test whether Python can achieve the same functionalities with different syntax. A specific use case to make this comparison was not done in this research, because it is difficult to find a neutral use case - a case that does not already predetermine a preference. However, it may well be that in the use cases chosen in this research, in order to achieve a certain result, such differences in syntax will arise, so those will indeed be analyzed.

- **Using services such as DFSMS, SDSF, DB2, ... :** The mainframe system is enormous, and Rexx (and Python) are used in various ways and for many use cases. Some of those use cases are carried out in this research, but unfortunately it is not possible to carry out all these use cases. These would be ideal use cases to carry out as a follow-up to this research.

4.3.2. Requirements

Below, both the requirements that will be used in the research and those that are not included are described. This is done to make the transparency as high as possible, and to clarify why certain requirements were or were not included in the research.
It quickly becomes clear that Python and Rexx differ quite a lot with regard to the chosen requirements. An advantage of one language is often a disadvantage of the other. Certain characteristics can also be viewed from a different angle, and an advantage can thus become a disadvantage and vice versa. A lot of nuance is therefore needed when interpreting these requirements.

Selected

The requirements from the requirements analysis that are most feasible and objectively measurable are the following:

- **Integration with tools on the mainframe:** It is generally known that Rexx has a head start over Python in this regard. This is, of course, because Rexx has been integrated with the mainframe since its creation. These tools, such as SDSF, RACF, DB2, CICS, TSO,... are crucial for being able to work with the mainframe. Rexx can work together with all these tools without any problems. Python can only do so with a few. Nevertheless, Python is catching up in this area. Projects such as the previously mentioned SEAR and ZOAU contribute to the integration of Python with the mainframe. However, this remains a pain point for Python due to the great diversity that exists on the mainframe.
- **Integration with tools outside the mainframe:** The entire world outside the mainframe is, of course, far too large to conduct a proper analysis of. For this research, the tools outside the mainframe are limited to tools that contribute to application development and tools that are needed for chain or package solutions.

Application development can involve GIT repositories in the cloud, or tools such as VS Code or IDz, but also quality checkers, analysis tools, and so on. All these tools largely exist alongside the mainframe and use, for example, SSH (Secure Shell, a widely used tool for obtaining a secure connection) to connect to the mainframe, something Python can work with very well. Rexx can also do this since USS, a mainframe environment, is used here. But Rexx still loses some functionalities in USS that it does have in TSO/ISPF.

Package solutions can involve managing incidents, password vaults such as CyberArk, or inventory lists such as CMDB (Configuration Management Database). All these things are maintained outside the mainframe, so the mainframe must be able to communicate with these tools. Communication with those tools mainly takes place using the HTTP protocol. That, again, is easy to do in Python, but much more complicated in Rexx.

- **Reusability of the code:** Reusability is understood to mean two things: (1) being able to use someone else's code via external packages, and (2) being able to reuse or modularize one's own code. For using someone else's code, there are thousands of packages in Python, which can be found on PyPI [1]. Reusing one's own code by placing a certain functionality in a separate code file, so that this code can be used multiple times, is also very easy in Python. In Rexx, both of these things are more complicated. A collection of packages does not exist as it does in Python, and in Rexx you cannot as easily get return values back when calling another Rexx program.

- **Security vulnerabilities:** Although the reusability of Python offers a clear advantage, there are also disadvantages attached to this. Python packages can be created by anyone, and can therefore be unsafe. In fact, bad actors are constantly looking for ways to attack companies via external packages that are downloaded. Although there are various ways to mitigate these risks, such as a company only allowing packages that have been tested and scanned, there is never 100% certainty that a particular package is safe. Rexx does not have this problem, since no new functionalities are being developed that could cause security issues.

- **Robustness of the code:** Again, the continuous progress of Python works against it here. To ensure that different domains of an organization can work at their own pace, virtual environments are often set up. Many Python packages are, after all, continuously updated and improved. In this way, different teams can use different versions of packages that correspond to the functionalities these teams need. A team may sometimes be forced to switch to a newer version of a package, for example because they start collaborating with

[1] https://pypi.org/

another team that uses a different version, and this can cause problems. A major example of such a renewal is when Python will be upgraded to version 4. Again, this is not a problem for Rexx, since no more changes are being made to the core of the language.

- **Support for the language:** One way to remedy the two previously mentioned disadvantages of Python, the security vulnerabilities and robustness, is the support that Python offers. Both the large vendors, the gigantic community, and the enormous amount of documentation and tools ensure that Python's support should not be underestimated. A package used by millions of people will also be reviewed by thousands of people to find and remove security vulnerabilities. The same applies to support when changing packages. There is always a very good chance that someone else has already experienced and solved your problem. In addition, there are many more tools that can help with developing and debugging Python than with Rexx. Although Rexx is also very well supported by vendors, and its documentation is easy to find and understand, it is clear that Python nevertheless has an advantage here thanks to its large community.

Not selected

These are some of the most frequently proposed requirements that were not chosen for this research. For each of the requirements, it is briefly explained what they mean and why they were not included in this research.

- **The number of people who will master the language now and in the future:** This probably refers to the previously discussed skills gap that exists in the mainframe field. There are more people who can work with Python than with Rexx. However, this is a requirement that is difficult to measure. One might assume that in the future more people will master Python than Rexx, but that is merely speculation. In addition, this prediction has no impact on the specific PoCs carried out in this research. That does not, of course, mean that this cannot have an impact on the future of the two languages, but rather that this requirement is difficult to measure in this research.
- **Performance and CPU usage:** Although this requirement would be excellent to include in a comparative study like this one, it is impossible to correctly compare the performance of the two languages without access to a full-fledged mainframe system. Since this research uses practice systems with relatively few resources available, this requirement cannot be measured.
- **Number of lines of code:** The code in this research was written by a student with relatively little programming experience, albeit with the help of an experienced systems programmer. However, it cannot be guaranteed that the code in this research was written in the most optimal way. In addition, certain

coding standards can cause this to vary from company to company. This requirement could potentially be merged with the requirement regarding the robustness of the language, which is included in this research.

5

Proof-of-Concept

5.1. PoC 1: Using an API

In this PoC, an attempt is made to make contact with an API via the mainframe. Contact is made with a private API running on the same mainframe, as well as the public API of open-meteo[1] to obtain the weather in Madrid. This makes it possible to see whether there is a difference when an API runs on a mainframe or not, and which parameters need to be changed for that.

5.1.1. Python

In the code of the Python files 5.1 and 5.2, the process is almost the same. Python's `requests` package is imported. This widely used package makes it possible to carry out API requests in a simple way. For the private API, a user ID and a password must be provided in order to log in correctly to the mainframe. An extra Python package is used for this. For the public API, certain parameters must be provided to obtain the information for a specific location (in this case Madrid). When using the requests package, it is automatically verified whether the server (the system being contacted) is secure. This is done using international standards and certificates. For the private API, this is manually disabled, because we know the system, and the certificates would make this example unnecessarily complex. After the request has been sent successfully or not, a response is received. If this response is good (with code 200), the results are displayed on the screen.

```
1    # The Python packages requests and getpass are imported
2    import requests
3    from getpass import getpass
```

[1] https://github.com/open-meteo/open-meteo

```
    # The credentials to log in are requested from the user and stored
    myCreds = ('','')

    if myCreds[0] == '':
        userid = input("Give your userid: ")
        myCreds = (userid,'')

    if myCreds[1] == '':
        passwd = getpass("Give your password: ")
        myCreds = (myCreds[0], passwd)

    # The URL to the private API of mainframeyin
    url = "https://mainframeyin:8092/LWWAPI/API/API"

    # The credentials are provided so that the login can be done
        correctly
    # It is not verified whether this API is secure, because we
        manage it ourselves
    response = requests.get(url, auth=myCreds, verify=False)

    if (response.status_code != 200):
        raise Exception(f"{response.status_code} - {response.text}")
    else:
        # We obtain the data in JSON format
        ApiList = response.json()

    # The different API names are displayed
    for api in ApiList['API']:
        print(api['APIName'])
```

listing 5.1: Making contact with a private API on the same mainframe with Python

```
    # The Python package requests is imported
    import requests

    # The coordinates for Madrid
    params = {
        "latitude": 40.4168,
        "longitude": -3.7038,
        "current_weather": "true",
```

```
        "temperature_unit": "celsius"
    }

    # The URL to the public API of open-meteo
    url = "https://api.open-meteo.com/v1/forecast"

    # No credentials are needed here, since the API is publicly
        available
    # We provide the parameters so that we get the weather for Madrid
    response = requests.get(url, params=params)

    # If there is an error, it is shown here
    if response.status_code != 200:
        raise Exception(f"{response.status_code} - {response.text}")
    else:
        # We obtain the data in JSON format
        weather_data = response.json()

    # We extract some data from the JSON
    current = weather_data['current_weather']
    temp = current['temperature']
    wind = current['windspeed']

    # The result is displayed
    print(f"The weather in Madrid:")
    print(f"Temperature: {temp} degrees Celsius")
    print(f"Wind speed: {wind} km/h")
```

listing 5.2: Making contact with a public API with Python

5.1.2. Rexx

In the code of the Rexx files 5.3 and 5.4, the Unix command `cURL` (Client URL) is used. This command is widely used for communication between systems and is therefore not specific to Rexx. Originally, the z/OS Client WET (Web Enablement Toolkit) was going to be used for Rexx, but this proved too complex for the relatively small example. Experienced system programmer Lionel Dyck gave this advice:
"I once looked at WET and, tbh(to be honest), gave up - seemed too complex for what it does. Using bpxwunix with standard unix tools seemed easier/simpler and more maintainable down the road. Just my $0.01 - ymmv(your mileage may vary)" (Lionel B. Dyck - IBM Z Champion on Discord)
The general process in Rexx is the same as in Python. A request is sent to an API

via a URL, with some parameters or credentials. In the Rexx code, the parameters and credentials must be added manually to the URL before it can be sent with the curl command. The curl command can be adjusted in various ways to fit the needs of the use case, similar to Python's requests package. A stem variable is provided, which works in a similar way to an array, into which the response will be placed. After checking for errors (rc = 0), the desired results can be displayed from the response using handy string manipulation.

```
/* REXX */

/* The URL to the private API */
url = 'https://mainframeyin:8092/LWWAPI/API/API'

/* Ask the user for the credentials to log in */
say "Give your userid: "
parse pull userid
myCreds.1 = userid

say "Give your password: "
parse pull passwd
myCreds.2 = passwd

/* We build the curl command:
-k : --insecure (similar to verify=False in Python)
-u : --user (for Basic Authentication)
*/
commando = "curl -k -u "myCreds.1":"myCreds.2" '"url"'"

/* Execute the Unix command */
rc = bpxwunix(commando,,response.,error.)

/* If there is an error, it is shown here */
if rc <> 0 then do
    say "Error executing curl. Return code:" rc
    do i = 1 to error.0
        say error.i
    end
exit 8
end

/* Processing of the JSON response */
```

```
    /* We search for the APINames in the JSON */
    ApiList = response.1

    say "List of APIs:"
    do while pos('"APIName":"', ApiList) > 0
        /* The different API names are displayed */
        parse var ApiList '"APIName":"' name '"' ApiList
        say name
    end

    exit 0
```

listing 5.3: Making contact with a private API on the same mainframe with Rexx

```
    /* REXX */

    /* We need to put the parameters in the url of the public API */
    url = 'https://api.open-meteo.com/v1/forecast?latitude=40.4168'
    || ,
    '&longitude=-
    3.7038&current_weather=true&temperature_unit=celsius'

    /* We build the Unix command with the public API of open-meteo */
    commando = "curl '"url"'"

    /* This is a Rexx command (bpxwunix) to execute a Unix command
    (curl).
    The response ends up in response., any errors in error. */
    rc = bpxwunix(commando,,response.,error.)

    /* If there is an error, it is shown here */
    if rc <> 0 then do
        say "Error executing curl. Return code:" rc
        do i = 1 to error.0
            say error.i
        end
    exit 8
    end

    /* We search specifically within the 'current_weather' section
    We cut the text from left to right
```

```
25      looking for temperature and windspeed */
26      parse var response.1 'current_weather":{' data '}' .
27      parse var data 'temperature":' temp ','
28      parse var data 'windspeed":' wind ','
29
30      say "The weather in Madrid:"
31      say "Temperature: " temp "degrees Celsius"
32      say "Wind speed:" wind "km/h"
33
34      exit 0
```

listing 5.4: Making contact with a public API with Rexx

5.1.3. Findings

First of all, there does not appear to be any difference between using an API on the (own) mainframe and using one outside the mainframe. This is because the connection happens via HTTP either way, so every connection is treated as a connection from outside. When creating the files and while executing the code, there were problems with the tags of the files. A different file tag is needed to correctly interpret and execute the Python and Rexx files. Such a file tag can be changed using the `chtag` command. For Python, `chtag -tc 819` was used, and for Rexx, `chtag -tc 1047`. To convert existing content of a file into a new file with the correct tag, the `iconv` command can be used.

In addition, to execute the Rexx files via USS, the command `chmod +x filename.rexx` had to be run. Aside from a possible dent in the user-friendliness of the mainframe via USS, the above findings have nothing to do with the following requirements.

Finally, it is also important to mention that the current way of requesting and providing the credentials is not the usual way. In a real automated script, there would be no user to enter the username and password; instead, certificates would typically be used to confirm the identity of the client. Because both Python and Rexx (with the help of curl) can handle this, that complexity has been left out of this PoC.

- **Integration with tools on the mainframe:** Rexx provides a very natural integration with the mainframe ecosystem through the use of `bpxwunix`, which allows direct use of the underlying USS. It should also be noted that using WET could potentially provide even better integration with Rexx. Python on the mainframe requires some specific actions, such as a correctly configured `venv` (virtual environment) and the installation of specific libraries. The latter can potentially cause difficulties if certain packages are not allowed. In this PoC, that was not a problem.

- **Integration with tools outside the mainframe:** Python is the clear winner here when using APIs. Thanks to the `requests` package, which is specifically designed for modern web communication and helps with things like SSL verification and JSON parsing, Python can communicate with an external API without any problems. Rexx has to rely on tools such as `curl` for external integration. Although this is effective, it requires more manual configuration of headers and parameters, which can complicate integration with complex APIs. In addition, string manipulation is needed to process the JSON reply, because Rexx itself cannot process JSON, but treats it as one large string.

- **Reusability of the code:** The Python code is considerably more reusable due to the use of standard packages and the ability to easily add logic to the code. In Rexx, the output is often heavily dependent on manual string manipulation (`parse var`) to extract data from a response. Although this is enormously powerful, it must be taken into account that, if the structure of the API output changes, the Rexx code also has to be adjusted in multiple places. In the current example, the output of the reply is so small that the string manipulation in Rexx is not overly complicated. However, if that output were to become larger, as is the case in real environments, the changes to Rexx's string manipulation would have an enormous impact on the reusability and user-friendliness of the language. Python is much more flexible in this regard because it can easily interpret the JSON output. In this way, the Python code depends on keywords from the JSON output, regardless of the size of the output.

  For both languages, one could choose to modularize certain parts of the code (placing pieces of code in other files to be called from the current file). Due to the limited size of the scripts, this is not necessary here. In that case, for Rexx, some additional setup would be needed to allow the scripts to communicate properly with each other.

- **Security vulnerabilities:** With the current implementation of the credentials, there is a significant security risk in the Rexx implementation because the credentials are passed as plain text in the `curl` command. This could lead to leaks in system logs or terminal history. Python offers safer alternatives such as the `getpass` module. However, as mentioned earlier, in a real environment the user's identity would be verified via certificates, so any security risks related to the credentials here are irrelevant. For both languages, an API request can be sent in a secure manner.

  Python makes use of external packages. These packages are downloaded from the internet beforehand. Since these packages (especially the requests package) are used enormously widely, we can assume that these packages are safe. In professional environments, these are also always thoroughly screened before actually being used. However, all these things do not mean

that security with regard to external packages offers 100% certainty.

- **Robustness of the code:** As mentioned earlier, if something changes in the structure of the API output, more things would need to change in Rexx than in Python, because Rexx uses string manipulation. However, the Rexx code will always work as the code describes, and would only need to be changed if the API output changes. On the other hand, changes may be necessary for Python if the underlying functionality or code of one of the used packages changes. These external packages are managed by entities outside the companies that use them. Any changes to such packages are therefore beyond the control of those companies. It is possible that at some point, the Python code will no longer do what is currently described in the code. In the case of widely used packages, such as the `requests` package used here, such code-breaking changes rarely or never happen (certainly not without the company being aware of it). Nevertheless, this is a fundamental difference between Rexx and Python, where the stability and predictability of Rexx is a major advantage.
- **Support for the language:** Little work is needed to write the Python code. The most complicated aspect is storing the credentials in a tuple. The packages are publicly available, with an abundance of examples and experts online providing information on how to write the code. For the Rexx code, on the other hand, there are fewer examples online. IBM provides official documentation on WET, which is far too complex for the simple example in this use case. The `curl` command is mainly recommended by users. And thanks to online documentation about this command, writing the Rexx code was feasible. Constructing the command by concatenating strings is not straightforward, and the string manipulation using `parse var` is, although powerful, less intuitive than JSON parsing in Python.

5.2. PoC 2: Generating JCL

In this PoC, a JCL job is generated with the help of skeletons. The purpose of this JCL is to copy the members of a PDS (Partitioned DataSet, the equivalent of a folder) to another PDS. A backup is thus created. The IEBCOPY utility of JCL is used for this purpose.

5.2.1. Python

In the Python code of 5.5, the jinja2 template in 5.6 is used. Some variables are defined beforehand. These variables are used to replace the placeholders (variables between double curly braces `{{ variable }}`) in the jinja2 template. The environment is set up and the jinja2 template is defined. Afterward, using the `render` method from the jinja2 package, the variables to be used can be provided to

replace the placeholders. Finally, the filled-in template is written to another file, `backupp.jcl`, in the same folder as the script. And that file is then ready to be executed.

```
    # The Python packages os and jinja2 are imported
    import os
    from jinja2 import Environment, FileSystemLoader

    # Defining the variables for the jinja substitution
    userid  = os.environ.get('USER')
    srcqual = 'MYLIB.SOURCE'
    tgtqual = 'MYLIB.BACKUPP'
    tgtdisp = 'MOD,CATLG,CATLG'

    # Set up the Jinja2 environment and load the skeleton file
    # from the same folder as the script
    file_dir = os.path.dirname(os.path.abspath(__file__))
    env      = Environment(loader=FileSystemLoader(file_dir))
    template = env.get_template('skeleton.j2')

    # Process the skeleton and substitute the variables
    jcl = template.render(userid=userid, srcqual=srcqual,
        tgtqual=tgtqual, tgtdisp=tgtdisp)

    # Write the generated JCL to a USS file in the same folder
    output_file = os.path.join(file_dir, 'backupp.jcl')
    with open(output_file, 'w') as f:
        f.write(jcl)
```

listing 5.5: Python code for using Jinja2 templates to generate JCL

```
    //{{ userid }} JOB 'TEST JOB',CLASS=A,MSGCLASS=N
    //S010    EXEC PGM=IEBCOPY
    //SYSPRINT DD SYSOUT=*
    //INDD     DD DISP=SHR,DSN={{ userid }}.{{ srcqual }}
    //OUTDD    DD DISP=({{ tgtdisp }}),
    //            DSN={{ userid }}.{{ tgtqual }},
    //            UNIT=SYSALLDA,SPACE=(TRK,(1,1)),
    //            DCB=(RECFM=FB,DSORG=PO,LRECL=80),DSNTYPE=LIBRARY
    //SYSIN    DD *
```

```
10      COPY INDD=INDD,OUTDD=OUTDD
11      /*
```

listing 5.6: The Jinja2 template used by Python

5.2.2. Rexx

In the Rexx code of 5.7, the skeleton in 5.8 is used via ISPF File Tailoring Services. Again, some variables are defined. In the case of Rexx and ISPF, the placeholders are the variables that start with a `'&'` in the skeleton. The only difference in the resulting JCL between Rexx and Python's JCL is that an 'X' is added to the job name here. This is due to the `USERID` being used as the job name in the JCL. If the job name of a JCL is the same as the user's user-id, ISPF asks for a character to be added to it. This has no further effect on how the JCL works. After defining the variables, commands are executed via an ISPF environment to generate the JCL. The placeholders are replaced and the result is first placed in a temporary dataset. It is then moved as member `BACKUPR` to a PDS called `GENJCL.JCL`. That member is replaced if it already exists. Finally, the references to datasets are released to return to the starting point. Aside from the variables, the largest part of the Rexx code is actually ISPF code, which is executed in an ISPF environment but sent from Rexx.

```
1   /* REXX */
2
3   /* Defining the variables that will be used in the skeleton */
4   SLIBDS  = 'GENJCL.ISPSLIB'
5   USERID  = USERID()
6   USERIDJ = USERID
7   if LENGTH(USERIDJ) < 8 then do
8       USERIDJ = USERIDJ || 'X'
9   end
10  SRCQUAL = 'MYLIB.SOURCE'
11  TGTQUAL = 'MYLIB.BACKUPR'
12  TGTDISP = 'MOD,CATLG,CATLG'
13
14  /* This indicates which environment the following commands
15  should be sent to. In this case that is ISPF */
16  ADDRESS ISPEXEC
17
18  /* The library containing our skeleton is added to any
19  existing ISPSLIB definition of ISPF */
20  "LIBDEF ISPSLIB DATASET ID("SLIBDS") STACK"
21
```

```
22      /* Open a temporary sequential dataset to store the JCL in. */
23      "FTOPEN TEMP"
24
25      /* Process the SKELETON and substitute all variables.
26      The generated output is written to the temporary dataset */
27      "FTINCL SKELETON"
28
29      /* Retrieve the name of the temporary dataset created by ISPF */
30      "VGET ZTEMPN SHARED"
31
32      /* Close the temporary dataset and finalize the generated JCL */
33      "FTCLOSE"
34
35      /* Initialize the temporary dataset for use by ISPF.
36      DATAID returns a reference (SRCDD) for use in later commands */
37      "LMINIT DATAID(SRCDD) DDNAME("ZTEMPN")"
38
39      /* Initialize the target dataset for use by ISPF.
40      DATAID returns a reference (TGTDD) for use in later commands */
41      "LMINIT DATAID(TGTDD) DATASET('"USERID".GENJCL.JCL')"
42
43      /* Copy the generated JCL from the temporary dataset to member
        BACKUPR
44      in the target dataset, and replace (REPLACE) it if it already
        exists */
45      "LMCOPY FROMID("SRCDD") TODATAID("TGTDD") TOMEM(BACKUPR) REPLACE"
46
47      /* Release the reference of the temporary dataset and the target
        dataset */
48      "LMFREE DATAID("SRCDD")"
49      "LMFREE DATAID("TGTDD")"
50
51      /* Remove our skeleton library from the ISPSLIB concatenation */
52      "LIBDEF ISPSLIB"
```

listing 5.7: Rexx code for using ISPF File Tailoring of JCL skeletons

```
1       //&USERIDJ JOB 'TEST JOB',CLASS=A,MSGCLASS=N
2       //S010    EXEC PGM=IEBCOPY
3       //SYSPRINT DD SYSOUT=*
4       //INDD     DD DISP=SHR,DSN=&USERID..&SRCQUAL
```

```
5     //OUTDD    DD DISP=(&TGTDISP),
6     //             DSN=&USERID..&TGTQUAL,
7     //             UNIT=SYSALLDA,SPACE=(TRK,(1,1)),
8     //             DCB=(RECFM=FB,DSORG=PO,LRECL=80),DSNTYPE=LIBRARY
9     //SYSIN    DD *
10    COPY INDD=INDD,OUTDD=OUTDD
11    /*
```

listing 5.8: The JCL skeleton used by ISPF File Tailoring with Rexx

Generated JCL

```
1     //YVG1 JOB 'TEST JOB',CLASS=A,MSGCLASS=N
2     //S010    EXEC PGM=IEBCOPY
3     //SYSPRINT DD SYSOUT=*
4     //INDD     DD DISP=SHR,DSN=YVG1.MYLIB.SOURCE
5     //OUTDD    DD DISP=(MOD,CATLG,CATLG),
6     //             DSN=YVG1.MYLIB.BACKUPP,
7     //             UNIT=SYSALLDA,SPACE=(TRK,(1,1)),
8     //             DCB=(RECFM=FB,DSORG=PO,LRECL=80),DSNTYPE=LIBRARY
9     //SYSIN    DD *
10    COPY INDD=INDD,OUTDD=OUTDD
11    /*
```

listing 5.9: The generated JCL file from both the Rexx and Python code

5.2.3. Findings

In this PoC, it was decided to run the Rexx part entirely in ISPF, and the Python part in USS. This is because these are the two environments in which both languages are mainly used and in which they work best. It is possible to make both environments work together, but that is unnecessary here.

Saving the JCL files is not necessary for the use case to function correctly. In a realistic scenario, the JCL could be executed immediately from the Rexx or Python code. In this PoC, they are saved simply so that the result can be analyzed.

In the JCL skeleton/template, commas are added at the end of certain lines. These commas allow a JCL command to continue on the next line. This is necessary because JCL code can be a maximum of 80 lines long (72 if the line ends in a comma), and we do not know in advance how long certain placeholders will be. Datasets are a maximum of 44 characters long, so if the placeholder for a dataset is placed on a separate line, there is certainly enough room.

Manually adjusting the Rexx and Python code to replace the variables for the placeholders is not user-friendly. Normally, skeletons/templates such as in this PoC are combined with a user-friendly way to dynamically adjust the variables, without having to dive into the code. This could potentially be an extension of the PoC, using ISPF screens for Rexx and, for example, the CLI (command line interface) with the help of the `argparse` command for Python. In this way, the variables could be adjusted dynamically.

- **Integration with tools on the mainframe:** Rexx again offers all the capabilities of the mainframe here. ISPF File Tailoring is a built-in mainframe service, and Rexx can address that service directly via `ADDRESS ISPEXEC`. No additional configuration is needed. Python, on the other hand, runs in USS, the Unix environment of the mainframe, and does not have direct access to ISPF services. To save the generated JCL in a z/OS dataset from Python, ZOAU is needed. Without that tool, writing to a dataset is not possible from USS. Because USS is still part of the mainframe, a JCL in USS can still be submitted. The integration with tools on the mainframe in this PoC therefore mainly depends on the preferred working environment, the traditional z/OS environment or the Unix environment. For Python, it is best to again set up a `venv` for managing the external packages, in this case `Jinja2`.
- **Integration with tools outside the mainframe:** In this PoC, there is no integration with tools outside the mainframe. Both the Rexx and Python code run entirely within the mainframe environment, respectively from ISPF and from USS. The generated JCL is stored on the mainframe itself.
- **Reusability of the code:** Both implementations make use of skeletons or templates, which benefits reusability. The variables in the code can easily be adjusted to copy other datasets. Both implementations could be further extended with a user interface, for example an ISPF screen for Rexx or a new script for Python, to make adjusting the variables even more user-friendly.
- **Security vulnerabilities:** In this PoC, there are no explicit security risks. For both languages, the generated JCL is stored locally on the mainframe, without any sensitive data being sent outside. A point of attention for Python is that this time 2 packages are used. The `os` package is, however, a built-in Python package, so it does not need to be installed alongside the installation of Python. However, another external package is used, namely `Jinja2`. This package is again widely used and well maintained, but external packages always carry a certain risk.
- **Robustness of the code:** Both implementations are robust for the described use. As long as the `SOURCE` dataset exists and the `BACKUP` dataset is correctly configured, the generated JCL will be executed correctly. One difference is

that the Rexx code depends on ISPF services, which are always available on a mainframe. The Python code, on the other hand, depends on Jinja2, an external package.

- **Support for the language:** Writing the Rexx code requires specific knowledge of ISPF File Tailoring. The official IBM documentation is available but fairly technical, and there are few practical examples to be found online. For Python and Jinja2, on the other hand, there is an abundance of documentation and examples available. Jinja2 is a widely used templating package that is also widespread outside the mainframe world, making it accessible for programmers with a Python background to get started with.

5.3. PoC 3: Using RACF

In this PoC, RACF is used to obtain a list of users who have been inactive for 3 months or longer. This list of users is converted into a sequential dataset, which can later be used, for example, in a batch job to restrict the access of these users.

5.3.1. Python

In the Python code of 5.10, SEAR is used to communicate with RACF. First, all users are retrieved. If SEAR does not return an error, a Python dictionary is returned, from which the last login date, `last_access_date`, of each user is retrieved. This is compared to today's date, and if the difference is greater than 90, the user is added to a list. Once all users have been added, they are written to a temporary file on USS. To make the list of users available as a dataset, a sequential dataset, `YVG1.RACFP.INACTIVE`, is created (this is first deleted if it already exists). If successful, the file with inactive users is copied using the `/bin/cp` command.

```
1   # A few Python packages, such as SEAR, are imported
2   from sear import sear
3   from datetime import datetime
4   import subprocess
5
6   # Use of datetime and a list for the inactive users
7   today = datetime.today()
8   inactive_users = []
9
10  # Retrieve all RACF users using SEAR
11  users = sear(
12  {
13      "operation": "search",
14      "admin_type": "user",
```

```
    },
    )

    # Loop over all users
    for i in range(0, len(users.result['profiles'])):
        user = users.result['profiles'][i]

        # Retrieve the user information for each user
        userdetail = sear(
        {
            "operation": "extract",
            "admin_type": "user",
            "userid": user,
        },
        )

        # Check whether the request succeeded
        if userdetail.result['return_codes']['sear_return_code'] ==
            0:

            # Check whether the last access date is available
            if 'base:last_access_date' in
                userdetail.result['profile']['base']:
                lastdate = userde-
                    tail.result['profile']['base']['base:last_access_date']
                # Convert the date from string to datetime object
                lastdate_parsed = datetime.strptime(lastdate,
                    '%m/%d/%y')

                # Calculate the difference in days
                diffdays = (today - lastdate_parsed).days

               # Add the user to the list if they have been inactive
                    for more than 90 days
                if diffdays > 90:
                    inactive_users.append(f"{user} : {diffdays}")

    # Write the inactive users to a temporary USS file
    with open('/tmp/inactive.txt', 'wb') as f:
        for line in inactive_users:
            f.write((line + '\n').encode('cp1047'))
```

```
51
52      dataset = "YVG1.RACFP.INACTIVE"
53
54      # Delete the old dataset and create a new sequential dataset
55      delete = subprocess.run(['tsocmd', f"DELETE '{dataset}'"],
            capture_output=True, text=True)
56      alloc = subprocess.run(
57      ['tsocmd', f"ALLOC DA('{dataset}') NEW CATALOG RECFM(FB)
            LRECL(80) BLKSIZE(8000) SPACE(1,1)
            TRACKS"],capture_output=True, text=True
58      )
59
60      # Check whether the creation succeeded
61      if alloc.returncode != 0:
62          print("Error creating dataset:", alloc.stderr)
63      else:
64          # Copy the temporary USS file to the dataset
65          copy = subprocess.run(['/bin/cp', '-P', 'RECFM=FB,LRECL=80',
                '/tmp/inactive.txt', f"//'{dataset}'"],
                capture_output=True, text=True)
66
67      # Check whether the copy succeeded
68      if copy.returncode != 0:
69          print("Error copying to dataset:", result.stderr)
70      else:
71          print(f"{len(inactive_users)} lines written to {dataset}")
```

listing 5.10: Python code for using RACF

5.3.2. Rexx

In the Rexx code of 5.11, the TSO environment is used to communicate with RACF. All users are listed and placed in the STEM variable `USERS.`. Then, for each user, the information from the `LU` command is stored again in a STEM variable `USERINFO.`. Using the `PARSE VAR` command, the attribute with the last login date is retrieved. This date is converted to a format that makes it easier to calculate with. If the difference with today's date is greater than 90, the user is added to the `INACTIVE.` STEM variable. Once all users have been processed, a new sequential dataset, `YVG1.RACFR.INACTIVE`, is created (the old one is deleted if it already exists), and all inactive users are written there.

```
/* REXX */
ADDRESS TSO

/* Today as number of days since 01/01/0001, and a list of
inactive users */
TODAY = DATE('B')
INACTIVE. = ''
INACTIVE.0 = 0

/* Retrieve all RACF users and store them in the USERS. stem */
x = OUTTRAP('USERS.')
'SR CLASS(USER)'
x = OUTTRAP('OFF')

/* Loop over all users */
DO I = 1 TO USERS.0
    USER = STRIP(USERS.I)
    /* Retrieve the information for each user via the LU command
*/
    x = OUTTRAP('USERINFO.')
    'LU' USER
    x = OUTTRAP('OFF')

    /* Look for the LAST-ACCESS line in the user information */
    DO J = 1 TO USERINFO.0
        IF LEFT(USERINFO.J, 13) = ' LAST-ACCESS='
        THEN DO
            /* Retrieve the last access date from the line */
            PARSE VAR USERINFO.J ' LAST-ACCESS=' LASTDATE '/' .

            /* Skip users with an UNKNOWN date */
            IF LASTDATE \= 'UNKNOWN'
                THEN DO
                /* Format the date like the TODAY variable */
                PARSE VAR LASTDATE LASTDATEYEAR '.' LASTDATEDAY
                LASTDATEJUL  = LASTDATEYEAR || LASTDATEDAY
                LASTDATEBASE = DATE('B', LASTDATEJUL, 'J')

                /* Calculate the difference in days */
                DIFFDAYS = TODAY - LASTDATEBASE
```

```
                    /* Add the user to the list if they have been
    inactive for more than 90 days */
                    IF DIFFDAYS > 90
                    THEN DO
                        COUNTER = INACTIVE.0 + 1
                        INACTIVE.COUNTER = USER ':' DIFFDAYS
                        INACTIVE.0 = COUNTER
                    END
                END
            END
        END
    END

    /* Delete the old dataset and create a new sequential dataset */
    DATASET = 'RACFR.INACTIVE'
    'DELETE' DATASET
    'ALLOC FI(OUTFILE) DA('DATASET') NEW CATALOG',
    'RECFM(F B) LRECL(80) BLKSIZE(8000) SPACE(1,1) TRACKS'

    /* Check whether the allocation succeeded */
    IF RC \= 0 THEN
    DO
        SAY 'ERROR: Could not allocate dataset, RC=' RC
        EXIT RC
    END

    /* Write all inactive users to the dataset */
    'EXECIO' INACTIVE.0 'DISKW OUTFILE (STEM INACTIVE. FINIS)'

    /* Check whether the write succeeded */
    IF RC \= 0 THEN
        SAY 'ERROR: Could not write to dataset, RC=' RC
    ELSE
        SAY INACTIVE.0 'records written to' DATASET

    /* Release the file */
    'FREE FI(OUTFILE)'
```

listing 5.11: Rexx code for using RACF

5.3.3. Findings

The use case for this PoC, retrieving users and adding them to a list based on their last login date, is not very realistic. There are several reasons why the use case would not work in a real production environment. There are edge cases, such as a user who has just been created but not yet used, that are not accounted for. In addition, there are already better tools for retrieving a user's login date. It must therefore be nuanced that this PoC is only intended to show the capabilities of Rexx and Python with RACF. One should look at the broader implication of this use case, namely that data can be retrieved with the help of RACF, and certain checks can be performed based on that data.

This use case consists of 2 parts. First, the use of RACF with Rexx and Python is demonstrated. Second, a dataset is created in which the data is stored. Both parts can be viewed separately from each other. These two parts were combined in order to compare even more aspects of integration with the mainframe without having to create a new PoC for each aspect.

In the Python code, the list of inactive users is temporarily stored in a USS file. For this, `encode('cp1047')` is used to ensure that the content of that file is readable for services on the mainframe. This is possibly not strictly necessary, but was added as a precaution due to earlier problems with file tags. This entire step - temporarily copying the data to a USS file in order to then copy it to a dataset - could potentially be replaced by functions in ZOAU that can complete this in one go.

- **Integration with tools on the mainframe:** Thanks to SEAR, Python can communicate with RACF without any problems. Via SEAR, Python makes use of the RACF callable service, which essentially means that it uses a local RACF program. In this way, Python can retrieve and use the data from RACF. In addition, Python can use the built-in `subprocess` functionality to execute TSO commands to delete and create a dataset, as well as to copy an existing USS file to it. For Rexx, all the above matters are self-evident. The TSO environment can be set up, and from there various RACF commands can be executed. Deleting and creating a dataset, as well as writing data to that dataset, can also be done without any problems.
- **Integration with tools outside the mainframe:** In this PoC, there is again no or hardly any integration with tools outside the mainframe. For Python, there is communication with the RACF callable service, but that happens locally on the mainframe. For Rexx, communication with RACF takes place via the TSO environment. Possible communication with RACF from outside could come from a company's HR department. Continuous input about employees joining, leaving, or going on vacation could result in many changes within RACF. Such automatic processing of data could be done with both Rexx and Python. This could be a possible extension of this use case.

- **Reusability of the code:** The Python code makes use of SEAR. This means that the RACF requests can easily be reused without any modifications. Deleting and creating the dataset, and copying the data, are also fully reusable. The Python code could technically be fully split up. For the Rexx code, various things are also reusable, such as the RACF commands. Processing the user info, where `PARSE VAR` is used to obtain the `LAST-ACCESS` attribute, is, although somewhat less intuitive, also reusable.
- **Security vulnerabilities:** For Python, 3 Python packages are imported. Only 1 of those 3 packages, SEAR, is an external package. SEAR is based on an official IBM project, RACFu, which has been discontinued. SEAR is developed by some well-known mainframers, such as Emma, who was interviewed for this research, and Bobby, the co-supervisor of this research. Therefore, we can assume that the SEAR package is safe. This conclusion, however, is based on a subjective opinion of the reliability of the package's developers. This does not make the package 100% safe.
- **Robustness of the code:** Again, Python's dependency on an external package such as SEAR plays a role here. The TSO environment used by Rexx will always be available. In the case of SEAR, a particular version of z/OS or Python could cause errors. Although this is well documented by the development team, it does put a dent in the robustness of the code. Python also has an additional vulnerability in the code, namely that a temporary USS file must first be created. This, however, is more a choice for convenience than a necessity.
- **Support for the language:** For both languages, in this case there is sufficient documentation and knowledge available to quickly have a working program. In the case of Python, SEAR's documentation is easy to find and very clear. The same applies to copying the USS file to a dataset. A brief question to the community and research into the `subprocess` package and the `/bin/cp` command gave a good result. For Rexx, more specific RACF knowledge is needed to execute certain commands and understand their output. Deleting and creating a dataset, and copying the data, are general Rexx knowledge and are also easy to find in the IBM documentation.

6

Results

In this chapter, the results of the proof-of-concept are discussed and analyzed. First, the circumstances and background of this research are discussed so that a nuanced picture of the results can be provided.

6.1. Background

In order to make a correct analysis of the findings of the various PoCs, it is necessary to mention several factors that may have influenced the results.

6.1.1. Limited comparison

First and foremost, this research was conducted by comparing two popular automation languages, Rexx and Python. However, there are other languages and technologies, such as Assembler or Ansible, that are also used on mainframes for automation. The choice of Python and Rexx was made due to the popularity of both languages, the ongoing discussion about these two languages within the mainframe community, and the time limit of this research. To conduct an ideal analysis regarding the future of automation on z/OS, all possible technologies, both old and new, should be compared.

6.1.2. System requirements

Secondly, an ideal study would utilize a system that operates like a production system, instead of the practice system used in this research. In that way, many objective requirements, such as performance, could actually be tested. Additionally, various other use cases could be executed on a full-fledged system, such as booting an LPAR. Related to this, few systems are identical. Mainframes can be extensively customized to meet the specific needs of a company. As a result, two environments are rarely the same. This can also affect the ability to reproduce certain use cases

in the exact same manner. However, the results from the use cases in this study should, in theory, not change much from system to system. Furthermore, there are certain prerequisites for executing these use cases. One of these requirements, for example, is that ZOAU is installed on the system. This allows Python to utilize various utilities on the mainframe. Since Rexx is installed by default on every system, Rexx does not face this issue.

6.1.3. Limitations of use cases

Thirdly, it is also important to note that this research is relatively small in scope. Only 3 use cases were executed, whereas there are numerous other possibilities within the mainframe environment. Although a general trend can be observed across the results of the executed use cases in this study, more use cases need to be performed to acquire a sufficient dataset from which more definitive conclusions can be drawn. A related limitation stems from the fact that, due to the limited number of use cases, this research only included use cases that were actually feasible for both languages to execute and would yield sound findings. An example of this is booting an LPAR. If we momentarily ignore that the system used was incapable of executing such a use case, the comparison itself would not provoke much discussion, since booting an LPAR can only be done using Rexx, or at least that remains by far the best method. Such a conclusion would not contribute much to the findings necessary to make this research valuable.

6.1.4. Out of scope

Finally, there are certain matters that fall outside the scope of the conducted research, but which can have an impact on the decision to choose one language over the other. These matters could be investigated in more detail in future research so that precise and accurate conclusions can be drawn.

- IBM utilizes a pricing mechanism where the use of Python - since Python can run on a zIIP (z Integrated Information Processor) - can lower workload costs compared to Rexx. The impact of this may vary significantly per system and company, and was not further investigated in this study.
- In the requirements analysis, it was already mentioned that 'the number of people who maintain the language now and in the future' is not used as a requirement in this study because this metric is difficult to measure and, additionally, does not influence how specific use cases are executed. However, this is an important requirement for companies to consider in the long term.
- Support for AI can play a role in choosing between the two languages. Since far more Python code is available online, AI models are often better trained on Python than on Rexx, where a lot of code resides on private corporate systems. This is an interesting topic to investigate, but it is currently still too unknown

and falls outside the scope of this research.

6.2. PoC Findings

6.2.1. Additional findings

The primary additional finding relates to file encoding on the mainframe. This is a well-known problem that many mainframers encounter, regardless of their experience level. Certain files can have a different file tag depending on where they are created, and the content of those files then follows that specific file encoding. If the wrong encoding is accidentally used, a new file must be created, and the content of the existing file must be converted to the correct encoding as well. Generally, Python code is written in Unicode and Rexx code in EBCDIC. To what extent this problem can be attributed to either language remains unclear, as the priority in this study was aimed at solving the encoding issues rather than evaluating the origin of the problem.
A few other findings include the following:

- A Rexx file requires a `chmod +x` command to be executed on USS.
- Rexx code must not exceed 80 lines. Often, TSO or ISPF commands are executed in Rexx that exceed 80 characters on a single line. In that case, the comma in Rexx can be utilized to continue on the next line.
- Python can only be used on USS. Unlike Rexx, which is also (and primarily) used outside of USS, Python can only be used within the USS environment of z/OS.
- VS Code, along with the open-source project Zowe - which provides the Zowe Explorer extension in VS Code for free - was used to develop most of the (Python) code. Thanks to Zowe Explorer, datasets, USS files, and even the output of JCL jobs can be viewed and edited directly within VS Code. Since VS Code is a popular code editor (especially among the younger generation), this can certainly improve the accessibility and user-friendliness of mainframes in general.

6.2.2. Requirements

Integration with tools on the mainframe:

- **Rexx** works as expected without any issues with other tools on the mainframe. Both TSO and ISPF commands can be executed from Rexx, through which various other utilities can be called. These utilities always remain available and present few to no problems.
- **Python** often requires the right environment and tools to use the same functionalities as Rexx. Python is only available within the USS environment of the

mainframe, where a virtual environment might need to be set up to install external packages. Python does not automatically have access to all of Rexx's functionalities and is often dependent on functionalities built by IBM, or by open-source projects such as SEAR. If these functionalities exist, they can be installed or set up, and Python can then often execute these tasks without issue. If they do not (yet) exist, it is usually impossible.

Integration with tools outside the mainframe:

- **Rexx** can often interact with tools outside the mainframe in one way or another. IBM tools exist for certain use cases, and for others, a solution can be cobbled together using existing tools like curl. Overall, however, these solutions feel complex and rigid. There are no clear guidelines or standardization, and manual work is required here and there in a manner that does not feel very robust (more on this under code robustness).

- **Python** features numerous built-in functionalities and can additionally leverage a massive collection of external libraries that provide even more capabilities. Communicating with APIs, which is increasingly necessary nowadays, can be done without any problem. Python can also work well with various formats from outside the mainframe, such as JSON. Python is built, and continuously expanded, to handle a vast array of tasks far beyond just the mainframe. This ensures that everything outside the mainframe proceeds smoother with Python.

Reusability of the code:

- **Rexx** requires a significant amount of work and manual adjustments when code is reused, depending on the use case. Modularization of the code is also not straightforward. The use of TSO and ISPF commands is the most reusable, as keywords can be adjusted beforehand. The complexity of the code regarding string manipulation certainly does not benefit reusability.

- **Python** is much more reusable thanks to the use of packages. Python already relies heavily on modularization through external packages, so Python functions can easily be modularized as well.

Security vulnerabilities:

- **Rexx** has the advantage here that nearly all necessary functionalities are available on the system by default. No external packages are downloaded, meaning there are very few risks when using Rexx.

- **Python** carries a risk greater than zero due to its packages. External Python packages can contain code that is harmful to the system or that could grant unwanted access to malicious actors. Thanks to the screening of such packages and their widespread use, most of these risks can be mitigated.

- Generally, having knowledge about the system and the code contained within potential external packages can prevent malicious code from being executed on the system.

Robustness of the code:

- **Rexx** features extremely powerful string manipulation, but this string manipulation only works if the external input does not change. If a specific (sequence of) input is expected and it changes, the Rexx code must be modified. The internal mechanics of the Rexx code itself will not change.

- For **Python**, the opposite applies. Python is more flexible regarding changes in input, but the internal workings of the Python code itself can change. This can happen due to updates to the external packages or tools used. In the worst-case scenario, an update to Python itself could occur. The extent of this impact is unknown, but it would likely give many teams a significant amount of extra work. In a less extreme scenario, different virtual environments can be set up within a company. If two environments need to be merged, this can also cause errors if different packages (or versions of packages) are used. If poorly managed, Python can cause a lot of additional work in this manner.

Support for the language:

- **Rexx** is natively supported by IBM on the mainframe. Rexx therefore has extensive IBM documentation available. Different people hold different opinions regarding IBM's documentation. The experience during this study was that the documentation is often too complex for what was needed. For instance, when using APIs, the choice was made not to use WET, even though it was designed to communicate with APIs. A major reason for this was that the documentation was too complex for the simple example being built. In terms of a community that can offer information and help, it is significantly smaller than Python's. However, there are a few experts present in the previously mentioned Discord community who can offer very deep expertise and assistance. Online examples are scarce or difficult to find.

- **Python** is increasingly supported by IBM, and thanks to open-source projects like Zowe and SEAR, support and integration are constantly improving. Python also has an enormous amount of information and examples online, ranging from simple tutorials to large-scale projects. This is partly due to its very large community, both inside and outside the mainframe space. Generally, it is easier to find information and help for Python than for Rexx.

7

Conclusion

This research attempted to answer the question of how Rexx and Python compare for automation on z/OS from both a technical and practical perspective. For the problem domain, the primary reasons for modernization on mainframes and the role of automation on mainframes were identified. The role of Rexx in automation was examined, and how Rexx became the most widely used scripting language. For the solution domain, Python and Rexx were compared based on predetermined requirements, and multiple PoCs were utilized to evaluate whether Python is already a worthy replacement for Rexx. Below, I discuss the answers to the five sub-questions that together answer the main research question.

7.1. Sub-question 1: Why is there a need for the modernization of mainframes?

Through the literature review (2), it was determined that modernization on the mainframe is primarily driven by both a growing shortage of workforce capable of working with mainframes, as well as the need to respond to the technological progress being made outside the mainframe platform. To keep old technology running on mainframes, new workforce members are required, and to use new technology, knowledge is necessary. However, if both problems are approached strategically, the situation can be reversed, and the solutions can stimulate each other. If a company utilizes new technologies, it can attract new talent, who in turn can assist with implementing new technologies on the mainframe, and so forth. Some parts of a system may not have a more modern alternative (yet). A renewal can also cost a lot of time and money, and during that time, the systems must keep running. It is therefore important for companies to find a healthy balance between continuing to modernize, in order to utilize new technologies, and attracting and training people into a workforce that has good background knowledge of the al-

ready existing older systems. In practice, both matters will likely run intertwined.

7.2. Sub-question 2: What does automation do on mainframes?

Based on the literature review (2) and the conducted interviews (8), it was concluded that automation plays a major role on the mainframe. Every day, various batch jobs are executed that serve numerous functions, such as generating reports, updating databases, and much more. These batch jobs work with enormous amounts of data and require little to no human interaction. These batch jobs are written in JCL and are, depending on the function of the job, usually executed at the beginning or the end of the workday. In addition to batch jobs, Rexx and Python are also heavily used to write scripts that perform all kinds of tasks. These scripts differ from JCL scripts because they are usually used to perform routine tasks that would otherwise be done manually by a programmer. The scripts themselves are still executed by a programmer and can have various functions. Examples of such functions include creating users, starting up or shutting down a system, connecting with APIs, and so on. The functions of scripts usually depend entirely on the specific job of the programmer.

7.3. Sub-questions 3 and 4: What are the advantages and disadvantages of Rexx and Python on z/OS?

From the results (6), it appears that Python and Rexx each have their own advantages and disadvantages. In the case of Rexx, `Integration with tools on the mainframe` and `Security vulnerabilities` are clear strengths, while Python scores strongly on `Integration with tools outside the mainframe` and `Reusability of the code`. For `Robustness of the code` and `Support for the language`, both languages have advantages and disadvantages, and the result depends on preference, knowledge, and the use case. An overview can be seen in figure 7.1 below.

Category	REXX	Python
Integration with tools on the mainframe	✓	×
Integration with tools outside the mainframe	×	✓
Reusability of the code	×	✓
Security vulnerabilities	✓	×
Robustness of the code	✓	✓
Support for the language	✓	✓

Figure 7.1: The results of the PoC's when comparing REXX and Python.

7.4. Sub-question 5: To what extent can Python already replace Rexx?

Although the comparison does not result in a clear winner, the findings of the use cases do point to a trend toward Python as more functionalities are added for Python on the mainframe. However, this does not mean that Rexx can simply be replaced. Rexx has a vast number of existing use cases that cannot easily be replaced by Python because they are so deeply interwoven with the mainframe. For new and modern use cases, where communication with the world outside the mainframe is becoming increasingly important, Python will often be the first choice.

7.5. Conclusion and recommendations

The research question of this study is as follows: How do Rexx and Python compare in terms of automation on z/OS from a technical and practical perspective? This study has shown that, in the current state of affairs, this question cannot be answered concisely and generally. The comparison between Rexx and Python must be made on a case-by-case basis. This is demonstrated by the different results in the PoCs. It is clear, however, that both languages consistently show certain strengths and weaknesses across different use cases. Rexx performs better in terms of integration with the mainframe system and security. Python scores strongly in terms of integration with the outside world and reusability. These general tendencies can aid in the choice of a language for a specific use case.

Further research with a better system and more time could provide the opportunity to involve more technologies, use cases, and requirements in the study. In this way, further research can be conducted into the future of automation on z/OS, and what role Python and Rexx can play in it. Examples include the use of other technologies, such as Ansible and Assembler, more use cases, such as booting an LPAR, and more requirements, such as CPU usage.

The recommendation for companies, as seen in figure 7.2, resulting from this re-

search is to modernize where possible by utilizing new technologies from outside the mainframe. This can contribute to attracting new workforce members, who in turn can be trained to maintain current systems and further assist in the development of new solutions within the mainframe ecosystem. This recommendation should however be taken with a grain of salt, as further research is required to gain more detailed results.

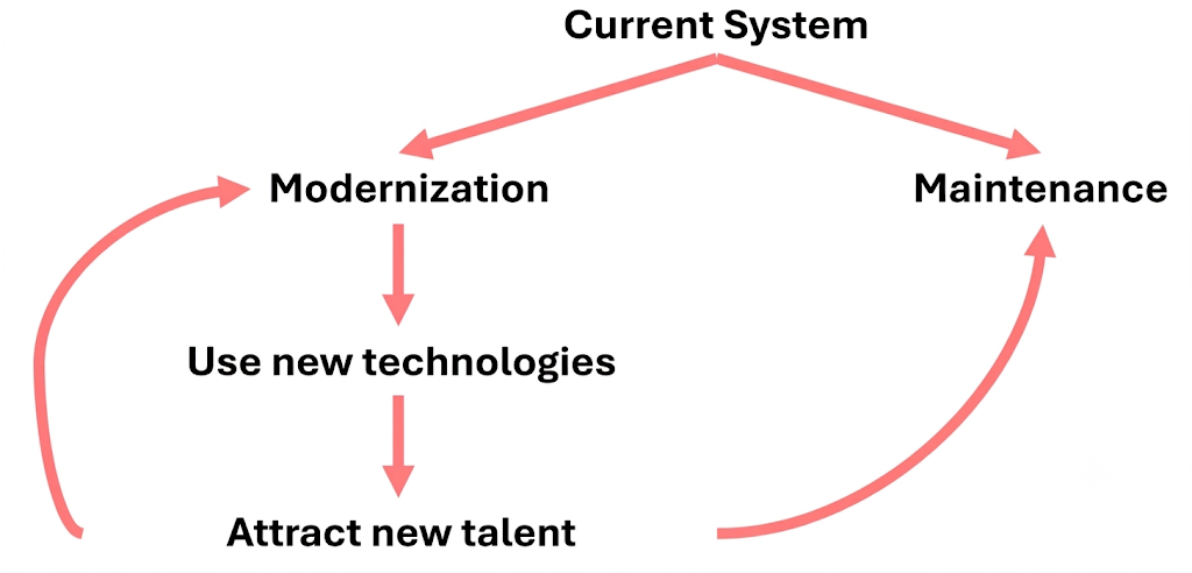

Figure 7.2: Recommendation loop for companies.

8

Interviews

8.1. Interview with Emma Skovgård

What are Rexx and Python used for on Mainframe in your point of view?

Rexx is used for z/OS Security because it's sometimes the only language available in those environments. Python is primarily used for automation, and for access to the USS environment. One-off Rexx scripts are popular, but you could do that with Python as well, like I do. A huge benefit of scripting in Python is access to the "argparse" command, which is incredibly powerful for handling command line arguments. In Rexx, you would have to hardcode those, which is a lot less flexible.

Since security (RACF) is your field of expertise, how do you see the use of both languages in security?

In my opinion Rexx isn't very good in RACF. Python has so many more built-in utilities that make it a lot easier to work with. I use Python for RACF clean ups and security automation, it's fairly good at handling large amounts of data. I was given the task of deleting 17000 RACF profiles at work and created a Python script with SEAR to delete them. This would not be possible with Rexx.

Many people are worried about the dependency problem with Python. So many external libraries can cause security issues, as well as the possibility for different teams using different versions of a package.

Having access to more tools isn't a bad thing. To put it bluntly: If you don't know how to use it, you shouldn't touch it. It's always best to avoid unnecessary dependencies. You should always check the Python standard library first. If it's not in the standard library, you could try to find an established external package. If there is a big community for it, chances are it's probably fine. In the worst case, you can build it yourself, which is a lot easier to do in Python than in Rexx. So even if there is no 3rd party packages, core Python is still great.

So how do you see the future of Automation (languages) on the mainfrane plat-

form?

In the next 20 years we won't see Rexx (or Clist, the predecessor of Rexx) go away. But if somebody wants to build something new, it will, or should be, in Python. Maintaining new languages is so much easier, so building infrastructure (automation) will be done using new languages and technologies. On top of that, data science/analysis, one-off financial reports and much more, are all done in Python. So there are so many more people who know Python than any of the older languages. If you want to attract new talent, you have to use the languages they know. Python is also more fully featured. It has all sorts of stuff built in that Rexx doesn't have. Even just converting variables is way easier in Python. Rexx in general is so much tougher to work with, which defeats the purpose of scripting. Lastly, Python used to be relatively slow, but it has gotten a lot faster.

What about access to Python on the z/OS platform?

API access is still a real problem for Python. ZOAU is constantly adding more though, it's growing. It'll probably be a while before we have all the Python or Linux functionalities in z/OS. But the good thing for companies adopting Python now and being early is, you won't have to do a lot when more and more functionality becomes available. You will already have the knowledge and experience to use it, while others will have to start from scratch when they want to adopt Python later on.

8.2. Interview with Bobby Tjassens

Vincent: What is your perspective on the history of automation on Z?

Bobby:

I started in 1998, roughly around the time when the mainframe was first predicted to be at the end of its life (which won't surprise you: that didn't happen). That was also the era when client-server computing was taking off. Before that, the mainframe was, for companies that had one, essentially the only thing they had. The mainframe stood completely on its own; everything that was needed ran on that same box. And although nobody had heard of automation yet, automation was already on the mainframe. By that I mainly mean job scheduling (JCL) and scripts for routine tasks (adding users, assigning permissions, handling messages in the log through system automation, compiling and link-editing application programs, etc.).

Client-server became the term for the exponential growth of X86 servers running applications and graphical user interfaces. Many of those applications needed data from the mainframe, which caused a major shift in online transaction traffic from terminal-based to service-oriented. Although the mainframe was no longer completely self-contained (because without the X86 components there was no application for the end user), the mainframe largely continued to do things "the mainframe way." This was a period of considerable arrogance on the mainframe side.

The people working with X86 experienced many problems that the mainframe had perhaps encountered years earlier, but had already solved in its own way. In my view, there was very little cross-working, at most in the form of cross-platform scheduling, where the mainframe was in charge.

With the arrival of server virtualization, the exponential growth of X86 increased by another factor. I think that what we understand as "automation" today was born around that time, simply because it became far too time-consuming to build, configure, and put every server into production manually. At the same time, a whole range of global communities was growing in which people shared knowledge online. The arrogance on Z remained, whereas I think we could already have learned a lot from one another back then. A lot of automation on X86 was not applicable to Z (we don't roll out dozens of new LPARs every week), but quite a bit of it was definitely relevant to Z as well. Think of test automation, automated deployment, observability (although it wasn't called that yet), etc.

Meanwhile, automation on Z of course didn't stand still. The mainframe happily continued to expand its use of scripting, including for things that X86 was working on in its own way. But up until what I consider this "point in time," this was done using the tools traditionally available on Z: JCL, REXX, and a handful of more exotic programming languages. "Point in time" in quotation marks, because of course something like this happens gradually. But at some point around then, a number of processes were "overtaken" by the outside world. Certain ideas, processes, and technologies had simply been worked on by so many more people outside Z, and those communities were simply so much larger, that certain global standards emerged naturally, and these were standards that were not born on Z.

From that point onward, the mainframe had to play catch-up in certain areas. Don't get me wrong: to this day, I believe the mainframe deserves its first-place position in large enterprises and that a lot of its potential remains untapped. But the world around the mainframe has fundamentally changed, and we are moving along with it (too) slowly.

In recent years, you can see that the roles have been reversed for certain processes and technologies, and the mainframe has had to follow. Think of centralized logging in Splunk, standard messaging such as Kafka, Git and Azure/DevOps (or something comparable), Ansible, OpenAPI, and now, more recently, AI. So, in my view, it goes beyond simply saying that the next generation is unfamiliar with the mainframe way of working. I am convinced that some solutions developed outside Z really are really better.

The effect on automation is that parts of the automation on Z need to integrate with the outside world. That can mean making scripts on Z callable from outside, but also the other way around: Z itself needs to make calls to systems outside the Z environment. In both cases, we have to conform to protocols and interfaces originating outside Z. At the same time, the mainframe has a reputation to uphold as a

stable platform on which (almost) everything continues to run, including systems developed decades ago. So you can't simply overhaul everything. This is where your research fits perfectly, because I think some companies struggle with exactly where to draw that line. Throwing away all your scripting and rewriting everything in Python is too costly (and impractical), so I suspect that rewriting tends to focus on the points of interaction with other platforms.

Sometimes I envy the new generation because of the rich environment they are entering. And sometimes I have compassion for them because I myself occasionally struggle with continuing to pile new technologies on top of what I already know, and that is where newcomers are starting today. That is why it is important that we, as the Z platform, do make certain choices in black and white and say goodbye to certain technologies that have become outdated, if only to keep the broad range of tools and languages manageable for the average person.

So we need standardization across Z shops, so that someone who has worked at mainframe site A can also work at B, C, and D.

Despite all that, I believe both REXX and Python should, for the time being, have a place in the standard toolkit.

9

Survey

9.1. Rexx vs Python comparative study - success criteria

I'm doing a comparative study between Rexx and Python. I'll be comparing both within the context of a couple of use cases. A few use cases that I have in mind, these are not set in stone (feel free to email me at vincent.gielen@student.hogent.be if you have other suggestions/feedback):

- IPL an LPAR (Rexx is favored)
- Use API's (Python is favored)
- Generate JCL (using Pythin Jinja2 vs Rexx with ISPF File Tailoring Services)
- Using RACF (details TBD)

9.1.1. Feel free to share your personal information (name, role, company, ...). This is not mandatory.

Enter your answer...

9.1.2. How long have you been working on Mainframes?

- Less than 5 years
- Between 5 and 10 years
- Between 10 and 25 years
- More than 25 years

9.1.3. What is your experience level with Python (on z/OS)?

- Beginner

- Intermediate
- Advanced
- Expert

9.1.4. What is your experience level with Rexx?

- Beginner
- Intermediate
- Advanced
- Expert

9.1.5. When comparing Python and Rexx, what are some requirements that you feel could be used? In other words, what (objective or subjective) criteria can be used to properly compare both? Some examples are (1) dependencies on external libraries or (2) lines of code, ...

Enter your answer...

9.1.6. Feel free to give any sort of comments, feedback, ... that can help me with my thesis.

Enter your answer...

Bibliography

Barnett, G. (2005). The future of the mainframe. *Ovum, october*, 38.

Blondeel, L. (2025, December 4). *Mainframe-specialisten hogent zijn zeer gegeerd in bedrijfswereld*. Retrieved January 26, 2026, from https://www.hogent.be/goedomweten/artikels/mainframe-specialisten-hogent-zeer-gegeerd-in-bedrijfswereld/

Cowlishaw, M. (1994). The early history of rexx. *IEEE Annals of the History of Computing*, *16*(4), 15–24. https://doi.org/10.1109/85.329753

Cowlishaw, M. F. (1984). The design of the rexx language. *IBM Systems Journal*, *23*(4), 326–335. https://doi.org/10.1147/sj.234.0326

Damme, D. V. (2025, November 28). *Mainframewereld richt ogen op gent* (T. DataNews, Ed.). Retrieved February 28, 2026, from https://datanews.knack.be/carriere/opleidingen/mainframewereld-richt-ogen-op-gent/

Docs, Z. (2026, January 21). *Zowe overview*. Retrieved March 1, 2026, from https://docs.zowe.org/stable/getting-started/overview/

Fosdick, H. (2005, March 11). *Rexx programmer's reference*. John Wiley & Sons.

Gilio, F. J. D. (2021, December 17). *Top 8 reasons for using python instead of rexx for z/os*. Retrieved January 26, 2026, from https://medium.com/theropod/top-8-reasons-for-using-python-instead-of-rexx-for-z-os-a6b67f8210a6

Goldberg, K. (2012). What is automation? *IEEE Transactions on Automation Science and Engineering*, *9*(1), 1–2. https://doi.org/10.1109/TASE.2011.2178910

IBM. (2026). *Ibm z xplore learning platform*. Retrieved February 28, 2026, from https://www.ibm.com/products/z/resources/zxplore

Jacobi, C. & Webb, C. (2020). History of ibm z mainframe processors. *IEEE Micro*, *40*(6), 50–58. https://doi.org/10.1109/MM.2020.3017107

Khadka, R., Shrestha, P., Klein, B., Saeidi, A., Hage, J., Jansen, S., van Dis, E. & Bruntink, M. (2015). Does software modernization deliver what it aimed for? a post modernization analysis of five software modernization case studies. *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 477–486. https://doi.org/10.1109/ICSM.2015.7332499

Learning, I. (2024). *Company history*. Retrieved February 28, 2026, from https://interskill.com/company-profile/company-history/#:~:text=2024,assessments,%20and%20labs%20now%20available.

Lindstrom, G. (2005). Programming with python. *IT professional*, *7*(5), 10–16.

Murphy, M. C., Sharma, A., Seay, C. & McClelland, M. K. (2010). Alive and kicking: Making the case for mainframe education. *Information Systems Education Journal*, *8*(44). Retrieved December 20, 2025, from https://eric.ed.gov/?id=EJ1146858

Ngo-Ye, T. L. & Choi, J. (2018). Teaching students mainframe skills for the niche market: An exploratory proposal.

Open Mainframe Project Modernization Working Group. (2024, December). *Mainframe programming languages, should i stay or should i go* (White Paper). Open Mainframe Project. Retrieved February 21, 2026, from https://openmainframeproject.org/wp-content/uploads/sites/14/2025/01/Mainframe-Programming-Languages-White-Paper.pdf?__hstc=160532258.4b44870ec4a577029c49e44b73bd3bee.1766016000205.1766016000206.1766016000207.1&__hssc=160532258.1.1766016000208&__hsfp=3006156910

Ousterhout, J. K. (1998). Scripting: Higher level programming for the 21st century. *Computer*, *31*(3), 23–30. https://doi.org/10.1109/2.660187

Perens, B. et al. (1999). The open source definition. *Open sources: voices from the open source revolution*, *1*, 171–188.

Perva, S. (2024, October 17). *Mainframers find harmony in discord, a community discussion platform* (SHARE, Ed.). Retrieved February 28, 2026, from https://blog.share.org/Trend-Watch-Article/mainframers-find-harmony-in-discord-a-community-discussion-platform

Phillips, B. K., Ryan, S., Harden, G., Guynes, C. S. & Windsor, J. (2013). Motivating students to acquire mainframe skills. *Proceedings of the 2013 Annual Conference on Computers and People Research*, 73–78. https://doi.org/10.1145/2487294.2487308

Powell, P. & Smalley, I. (2024, February 9). *What are batch jobs?* IBM. Retrieved February 23, 2026, from https://www.ibm.com/think/topics/batch-jobs

Prechelt, L. (2000). An empirical comparison of seven programming languages. *Computer*, *33*(10), 23–29. https://doi.org/10.1109/2.876288

Project, M. O. E. (2025). *Mainframe events and conferences 2025*. Retrieved February 28, 2026, from https://open-mainframe-project.gitbook.io/mainframe-open-education-project/additional-mainframe-resources/mainframe-events-and-conferences-2025

Project, O. M. (2025). *Open mainframe projectannual report*. Retrieved March 1, 2026, from https://openmainframeproject.org/wp-content/uploads/sites/14/2026/01/Open-Mainframe-Project-Annual-Report-1.pdf

Redwerk. (2024, October 10). *The pros and cons of python programming language*. Retrieved March 2, 2026, from https://redwerk.com/blog/pros-and-cons-of-python/#:~:text=As%20for%20disadvantages,%20Python%20can,intensive%20than%20some%20other%20languages.

Sadavarte, R. & Prabhu, V. (2022). Rexx vs python: A comparative study, 189–197. Retrieved December 28, 2025, from https://soe.rku.ac.in/conferences/data/23_1916_ICSET%202022.pdf

Sagers, G., Ball, K., Hosack, B., Twitchell, D. & Wallace, D. (2018). The mainframe is dead. long live the mainframe! *AIS Transactions on Enterprise Systems*, *2*(1). https://www.aes-journal.com/index.php/ais-tes/article/view/6

Sanglier, P.-A., Lewandowski, P. & Klumb, F. (2022, November). *First experience on python development for survey software, advantages and drawbacks*. CERN. Geneva. Retrieved December 31, 2025, from https://cds.cern.ch/record/2849051

Singh, D. (2025, May 12). *Leveraging ibm python and zoau for agile and scalable system programming*. Retrieved March 1, 2026, from https://medium.com/@div25singh/leveraging-ibm-python-and-zoau-for-agile-and-scalable-system-programming-bc861f2eee0f

Sochova, Z. & Kunce, E. (2012). A story about a dinosaur called mainframe and a small fly agile. *2012 Agile Conference*, 74–78. https://doi.org/10.1109/Agile.2012.11

Spain, S. C. (2025, June 19). *Santander completes the digitalization of its technology infrastructure in spain with the deployment of gravity*. Retrieved March 1, 2026, from https://www.santander.com/en/press-room/press-releases/2025/06/santander-completes-the-digitalization-of-its-technology-infrastructure-in-spain-with-the-deployment-of-gravity

Stine, K. (2022). *Ibm z16 technical overview* (IBM zSystems Tech Bytes) [Presented by the Washington Systems Center]. IBM Washington Systems Center. Retrieved February 21, 2026, from https://www.ibm.com/support/pages/system/files/inline-files/z16%20Technical%20Overview_0.pdf

Susnjara, S. & Smalley, I. (2024, April 8). *What is a mainframe?* IBM. Retrieved February 24, 2026, from https://www.ibm.com/think/topics/mainframe

Thota, R. C. (2020). End-to-end automation of multi-cloud deployments using terraform, ansible, and kubernetes. *International Journal of Innovative Research in Engineering and Multidisciplinary Physical Sciences*, *8*(4), 1–10. https://doi.org/10.37082/ijirmps.v8.i4.232184

University, N. I. (2026). *Corporate training*. Retrieved February 28, 2026, from https://www.cs.niu.edu/careers/training/

Vanaparthi, N. R. (2025). The roadmap to mainframe modernization: Bridging legacy systems with the cloud. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, *11*(1), 125–133. https://doi.org/https://doi.org/10.32628/CSEIT25111214

Winkler, T. & Flatscher, R. G. (2023). Cognitive load in programming education: Easing the burden on beginners with rexx. *Central European Conference on Information and Intelligent Systems*, 171–178.

Zlatanov, N. (2016). The data center evolution from mainframe to cloud. *IEEE Computer Society*.